

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a5ea2f43dc65f6672.jsonl`  
- SHA-256: `a72fb06cb653e79a6d31eb2f19ec6f89fb0143673bcc58186d6b2b32bdce73a1`  
- Source modified: `2026-06-10T16:25:50+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:13:27.751Z)

You are doing a DEEP correctness review of gmail-attachments-to-gdrive (C:/Users/avidu/Projects/gmail-attachments-to-gdrive), a Google Apps Script project written in TypeScript, bundled with esbuild, tested with Jest. It archives Gmail attachments to the user's own Drive on time-based triggers. Key mechanics: date-window backfill sync plus an opt-in hybrid incremental sync via the Gmail advanced service history.list with a stored startHistoryId (404 = expired cursor, falls back to window re-scan); a LockService script lock against concurrent runs; trigger fan-out with a cap; a bounded FIFO of processed message ids persisted in user properties; per-day file-create budget (MAX_FILES_ADD_PER_DAY=240, self-imposed); 5-minute MAX_RUNTIME_MS budget against the 6-minute hard kill; a multi-user-capable web app (executeAs USER_ACCESSING); opt-in daily summary email.

Read EVERY file under src/ completely (config.ts, contentTypes.ts, drive.ts, entry.ts, gmailHistory.ts, historySync.ts, logger.ts, naming.ts, notify.ts, processed.ts, query.ts, quota.ts, state.ts, stats.ts, sync.ts, ui.ts, types/gmailAdvanced.d.ts), plus appsscript.json, esbuild.config.mjs, jest.config.js, .github/workflows/ci.yml, and the tests under tests/ where needed to confirm intent.

Hunt specifically for (Apps Script semantics matter ? reason about the GAS runtime, not generic TS):

- PropertiesService limits: 9KB per value, 500KB total per store ? can the processed-ids FIFO, stats, or history cursor blow these? What happens on overflow?
- History sync: is the mailbox historyId captured BEFORE processing (mid-run mail)? Off-by-one on historyId resumption (history.list returns changes AFTER the given id)? Duplicate/missed message handling across the backfill-to-incremental handover? Pagination of history.list? labelId filtering correctness?
- Date-window logic: timezone of Gmail search operators (after:/before: are in the user's local tz? PT?), boundary overlap/gap between windows, isToday determination.
- Locking/triggers: lock acquisition failure path, trigger leak when an execution dies at the 6-min kill (no finally), trigger cap pruning correctness, self-healing claims.
- 'Mark done only on clean completion': verify the invariant holds on every path (daily cap hit, transient Drive error, runtime budget exhausted mid-message, attachment-level failure).
- FIFO eviction: can an evicted id be re-processed and create duplicate Drive files? Interaction with Drive getFilesByName dedup probe; naming collisions (same filename, different attachments).
- Quota accounting: where Gmail read calls are counted vs actually consumed; the 240/day Drive budget reset logic across timezones/DST.
- Multi-user web app: state isolation between users, setup input validation, what a second user's first run does.
- Error handling: swallowed errors, notify/email failure paths, logger PII (subjects/filenames in logs ? logs go to Cloud Logging! is that egress per their own privacy claims?).
- Build/CI: esbuild bundling pitfalls for GAS (global fn exposure for triggers), clasp deploy gating.

Report ONLY real, defensible findings with exact file:line and quoted code evidence; honest severities; include genuinely-worth-fixing medium/low items. Structured output only.

2. user (2026-06-10T16:13:32.743Z)

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\description
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\applypatch-msg.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\commit-msg.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\fsmonitor-watchman.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\post-update.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-applypatch.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-commit.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-merge-commit.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-push.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-rebase.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-receive.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\prepare-commit-msg.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\push-to-checkout.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\sendemail-validate.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\update.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\info\exclude
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.idx
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.pack
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.rev
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\refs\remotes\origin\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\packed-refs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\refs\remotes\origin\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.clasp.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.claspignore
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\config
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\refs\heads\master
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\refs\heads\master
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.github\workflows\ci.yml
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.gitignore
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\appsscript.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\docs\ARCHITECTURE.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\docs\VISION.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\esbuild.config.mjs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\eslint.config.mjs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\LICENSE
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\PRIVACY.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\README.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\ROADMAP.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\html\index.html
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\jest.config.js
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\package-lock.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\package.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\config.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\contentTypes.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\drive.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\entry.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\historySync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\logger.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\naming.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\processed.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\query.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\quota.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\stats.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\types\gmailAdvanced.d.ts

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\ui.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\auditability.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\contentTypes.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\historySync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\naming.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notify.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\processed.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\quota.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\index
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\setup.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\stats.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tsconfig.json

3. user (2026-06-10T16:13:34.902Z)

/c/Users/avidu/Projects/gmail-attachments-to-gdrive/.clasp.json
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/.claspignore
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/.github/workflows/ci.yml
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/.gitignore
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/appsscript.json
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/docs/ARCHITECTURE.md
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/docs/VISION.md
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/esbuild.config.mjs
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/eslint.config.mjs
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/html/index.html
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/jest.config.js
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/LICENSE
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/package-lock.json
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/package.json
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/PRIVACY.md
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/README.md
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/ROADMAP.md
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/config.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/contentTypes.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/drive.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/entry.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/gmailHistory.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/historySync.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/logger.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/naming.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/notify.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/processed.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/query.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/quota.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/state.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/stats.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/sync.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/types/gmailAdvanced.d.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/src/ui.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/auditability.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/contentTypes.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/drive.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/gmailHistory.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/historySync.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/naming.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/notify.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/processed.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/query.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/quota.test.ts

```
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/setup.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/stats.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/sync.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/syncIncremental.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tests/ui.test.ts
/c/Users/avidu/Projects/gmail-attachments-to-gdrive/tsconfig.json
```

4. user (2026-06-10T16:13:40.576Z)

```
1      /*
2      * The MIT License (MIT)
3      * Copyright (c) 2013 Avi Dullu
4      */
5
6      /** Top-level Drive folder that holds every synced attachment. */
7      export const GMAIL_ATTACHMENT_FOLDER = "GmailAttachments";
8
9      /** Public project / support page shown in the setup UI. */
10     export const SUPPORT_URL =
11         "https://github.com/avidullu/gmail-attachments-to-gdrive";
12
13     /** Privacy policy shown in the setup UI (and the OAuth consent screen). */
14     export const PRIVACY_POLICY_URL =
15         "https://github.com/avidullu/gmail-attachments-to-gdrive/blob/master/PRIVACY.md";
16
17     /**
18      * Apps Script enforces a daily quota on file creation. We cap ourselves below
19      * it so a single run can never exhaust the account's allowance.
20      */
21     export const MAX_FILES_ADD_PER_DAY = 240;
22
23     // Time constants, in milliseconds.
24     export const ONE_MINUTE = 60_000;
25     export const ONE_DAY = 86_400_000;
26     export const THIRTY_DAYS = 2_592_000_000;
27     export const ONE_YEAR = 31_622_400_000;
28     export const SIX_HOURS = 21_600_000;
29
30     /**
31      * Wall-clock budget for a single sync execution. Apps Script kills consumer
32      * scripts at 6 minutes, so we stop at 5 to leave margin for the final
33      * bookkeeping and trigger scheduling.
34      */
35     export const MAX_RUNTIME_MS = 5 * ONE_MINUTE;
36
37     /** Defensive upper bound on how many sync triggers may exist at once (limit is 20). */
38     export const MAX_SYNC_TRIGGERS = 12;
39
40     // Keys used in PropertiesService.getUserProperties().
41     export const PROP_FOLDER = "f";
42     export const PROP_LAST_SYNCED = "l";
43     export const PROP_FILES_ADDED_TODAY = "fa";
44     export const PROP_DEBUG = "dbg";
45     export const PROP_LAST_ERROR = "err";
46     export const PROP_PROCESSED_IDS = "pid";
47     export const PROP_RUN_STATS = "stats";
48     export const PROP_SUMMARY_LAST_SENT = "sumsent";
49     export const PROP_HISTORY_ID = "hid";
50
51     /** Flip to true to emit verbose Logger output during development. */
52     export const ENABLE_DEBUGGING = false;
53
54     /**
55      * Hybrid incremental sync. OFF by default so the proven date-window backfill
56      * remains the sole code path; flip to true to roll out the cheaper Gmail
```

```

57     * `history.list` steady state. When on, once a backfill catches up the current
58     * mailbox `historyId` is captured and subsequent runs process only newly-added
59     * messages, falling back to a date-window re-scan if the id expires (HTTP 404).
60     * Requires the Gmail advanced service (declared in appsscript.json); no extra
61     * OAuth scope ? `gmail.readonly` already covers history reads.
62     */
63     export const ENABLE_INCREMENTAL_SYNC = false;
64
65     /**
66     * Daily summary email. OFF by default ? enabling it requires the
67     * `script.send_mail` OAuth scope (a re-consent on next deploy). When enabled,
68     * set SUMMARY_EMAIL_RECIPIENT to the destination address (empty = the
69     * effective user's own address).
70     */
71     export const ENABLE_SUMMARY_EMAIL = false;
72     export const SUMMARY_EMAIL_RECIPIENT = "";
73

```

5. user (2026-06-10T16:13:41.024Z)

```

1     /**
2     * The MIME types we are willing to copy into Drive. Anything not on this list
3     * is skipped. Replaces the old BucketsLib Set with a native Set.
4     */
5     export const ALLOWED_CONTENT_TYPES: ReadonlySet<string> = new Set<string>([
6         // Text
7         "text/plain",
8         "text/csv",
9         "text/xml",
10        "text/html",
11        "text/cmd",
12        "text/javascript",
13        // Images
14        "image/jpeg",
15        "image/png",
16        "image/gif",
17        "image/webp",
18        "image/bmp",
19        "image/tiff",
20        "image/heic",
21        "image/heif",
22        "image/svg+xml",
23        // Application
24        "application/pdf",
25        "application/javascript",
26        "application/json",
27        "application/xml",
28        "application/zip",
29        "application/msword",
30        "application/vnd.ms-powerpoint",
31        "application/x-shockwave-flash",
32        "application/vnd.ms-project",
33        "application/vnd.ms-works",
34        "application/vnd.ms-outlook",
35        "application/vnd.ms-excel",
36        "application/postscript",
37        "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
38        "application/vnd.openxmlformats-officedocument.presentationml.presentation",
39        "application/vnd.mozilla.xul+xml",
40        "application/vnd.oasis.opendocument.spreadsheet",
41        "application/vnd.oasis.opendocument.presentation",
42        "application/vnd.oasis.opendocument.text",
43        // Audio
44        "audio/mpeg",
45        "audio/mp4",

```

```

46     "audio/mid",
47     // Video
48     "video/mpeg",
49     "video/mp4",
50     "video/x-flv",
51     "video/x-ms-wmv",
52     "video/x-msvideo",
53     "video/quicktime",
54 ];
55
56 /**
57  * Normalises a raw `Content-Type` header to its bare media type: strips any
58  * parameters (`; charset=utf-8`, `; name="x.pdf"`), trims, and lower-cases. The
59  * Gmail/MIME value often carries parameters or odd casing, which a plain
60  * `Set.has` lookup would miss.
61  */
62 function normalizeContentType(type: string): string {
63     return type.split(";")[0].trim().toLowerCase();
64 }
65
66 export function isAllowedContentType(type: string): boolean {
67     return ALLOWED_CONTENT_TYPES.has(normalizeContentType(type));
68 }
69

```

6. user (2026-06-10T16:13:41.989Z)

```

1     import { GMAIL_ATTACHMENT_FOLDER } from "./config";
2     import { isAllowedContentType } from "./contentTypes";
3     import { getAttachmentName, getFolderNameForStructure } from "./naming";
4     import { logDebug, logError } from "./logger";
5
6     type Folder = GoogleAppsScript.Drive.Folder;
7
8     /**
9      * Anything that can hold folders: both `DriveApp` (the root) and a `Folder`
10     * expose `getFolders()` / `createFolder()`, so we accept either structurally.
11     */
12     interface FolderContainer {
13         getFolders(): GoogleAppsScript.Drive.FolderIterator;
14         createFolder(name: string): Folder;
15     }
16
17     export interface UpdateResult {
18         /** Number of files actually written to Drive in this call. */
19         addedFiles: number;
20         hasError: boolean;
21         /** Count of attachment-level errors in this call. */
22         errors: number;
23         /** Most recent error message, if any. */
24         lastError: string;
25         /** Gmail message ids fully handled in this call (for cross-run dedup). */
26         processedIds: string[];
27     }
28
29     /** Returns the named child folder, creating it if it does not exist yet. */
30     export function getOrCreateFolderInFolder(
31         parent: FolderContainer,
32         name: string,
33     ): Folder {
34         const folders = parent.getFolders();
35         while (folders.hasNext()) {
36             const folder = folders.next();
37             if (folder.getName() === name) {
38                 logDebug("Found existing folder " + name);

```

```

39         return folder;
40     }
41 }
42 logDebug("Creating new folder " + name);
43 return parent.createFolder(name);
44 }
45
46 /**
47  * Resolves the destination sub-folder for an attachment, creating it under
48  * `head` and caching it in `existingFolders`. Returns null when the chosen
49  * structure yields no folder (caller falls back to the head folder).
50  */
51 function resolveFolder(
52     head: Folder,
53     existingFolders: Map<string, Folder>,
54     structure: string,
55     contentType: string,
56     date: Date,
57 ): Folder | null {
58     const name = getFolderNameForStructure(structure, contentType, date);
59     if (name === "") {
60         return null;
61     }
62     const cached = existingFolders.get(name);
63     if (cached) {
64         return cached;
65     }
66     const created = head.createFolder(name);
67     existingFolders.set(name, created);
68     return created;
69 }
70
71 /**
72  * Copies every allowed, not-yet-present attachment from the given threads into
73  * the GmailAttachments folder, organised per the user's chosen structure.
74  * Stops once `maxToAdd` files have been written.
75  *
76  * `alreadyProcessed` is the set of Gmail message ids handled in previous runs;
77  * such messages are skipped wholesale, avoiding a Drive `getFilesByName` probe
78  * per attachment when an overlapping date window is re-scanned. Ids fully
79  * handled here are returned in `processedIds` for the caller to persist.
80  */
81 export function updateDrive(
82     threads: GoogleAppsScript.Gmail.GmailThread[],
83     structure: string,
84     maxToAdd: number,
85     alreadyProcessed: ReadonlySet<string> = new Set(),
86 ): UpdateResult {
87     const head = getOrCreateFolderInFolder(DriveApp, GMAIL_ATTACHMENT_FOLDER);
88
89     // Pre-load existing sub-folders so we reuse rather than duplicate them.
90     const existingFolders = new Map<string, Folder>();
91     const currentFolders = head.getFolders();
92     while (currentFolders.hasNext()) {
93         const folder = currentFolders.next();
94         existingFolders.set(folder.getName(), folder);
95     }
96     logDebug("Already present folders: " + existingFolders.size);
97
98     const addedFiles = new Map<string, Folder>();
99     const processedIds: string[] = [];
100     let addMore = maxToAdd > 0;
101     let hasError = false;
102     let errors = 0;
103     let lastError = "";

```

```

104
105     for (const thread of threads) {
106         if (!addMore) break;
107         for (const message of thread.getMessages()) {
108             if (!addMore) break;
109             const messageId = message.getId();
110             // Skip messages we fully handled on a previous run.
111             if (alreadyProcessed.has(messageId)) {
112                 logDebug("Skipping already-processed message " + messageId);
113                 continue;
114             }
115             let messageFailed = false;
116             const from = message.getFrom();
117             // GmailMessage#getDate is typed as the stripped Base.Date interface but is
118             // a real JS Date at runtime.
119             const date = message.getDate() as unknown as Date;
120             const attachments = message.getAttachments();
121             let lastHandled = -1;
122             for (let a = 0; a < attachments.length && addMore; a++) {
123                 const attachment = attachments[a];
124                 lastHandled = a;
125                 const contentType = attachment.getContentType();
126                 const fileName = getAttachmentName(attachment.getName(), from, date);
127
128                 if (!isAllowedContentType(contentType) || addedFiles.has(fileName)) {
129                     logDebug("Rejected file of type: " + contentType);
130                     continue;
131                 }
132
133                 let folder = resolveFolder(
134                     head,
135                     existingFolders,
136                     structure,
137                     contentType,
138                     date,
139                 );
140                 if (folder == null) {
141                     // Unknown structure: drop it in the top-level folder for recovery.
142                     logError("Could not resolve directory; using top-level folder.");
143                     folder = head;
144                 }
145
146                 try {
147                     // Skip if a file with this name already exists in the folder.
148                     if (folder.GetFilesByName(fileName).hasNext()) {
149                         continue;
150                     }
151                     const blob = attachment.copyBlob();
152                     blob.setName(fileName);
153                     const file = folder.createFile(blob);
154                     file.setDescription(
155                         "Created from a Gmail Attachment from " +
156                         from +
157                         " sent on date " +
158                         date,
159                     );
160                     addedFiles.set(fileName, folder);
161                     Utilities.sleep(100);
162                     if (addedFiles.size === maxToAdd) {
163                         addMore = false;
164                     }
165                 } catch (e) {
166                     logError("Attachment copy failed (message " + messageId + "): " + e);
167                     hasError = true;
168                     messageFailed = true;

```



```

169         errors++;
170         lastError = String(e);
171         Utilities.sleep(1000);
172     }
173 }
174 // Mark the message done only if we handled all its attachments cleanly.
175 // If we stopped early on the daily cap (lastHandled short of the end) or
176 // hit a transient failure, leave it unmarked so the next run retries it.
177 const completed = !messageFailed && lastHandled === attachments.length - 1;
178 if (completed) {
179     processedIds.push(messageId);
180 }
181 }
182 }
183
184 return { addedFiles: addedFiles.size, hasError, errors, lastError, processedIds };
185 }
186

```

7. user (2026-06-10T16:13:42.088Z)

```

1  /*
2   * Bundle entry point. esbuild compiles this (with all imports) into a single
3   * IIFE assigned to the `GmailToDrive` global. A footer in esbuild.config.mjs
4   * then declares top-level `doGet` / `processInput` / `syncUserDrives` functions
5   * that delegate here, so the Apps Script runtime can resolve them by name for
6   * the web app, `google.script.run`, and time-based triggers.
7   */
8   export { doGet, processInput } from "./ui";
9   export { syncUserDrives } from "./sync";
10

```

8. user (2026-06-10T16:13:47.355Z)

```

1   import {
2       collectAddedMessageIds,
3       isHistoryExpired,
4       nextHistoryId,
5       type HistoryListResponse,
6   } from "./historySync";
7   import { logDebug } from "./logger";
8
9   type GmailThread = GoogleAppsScript.Gmail.GmailThread;
10
11   /** A bounded number of history pages to walk in a single run. */
12   const MAX_HISTORY_PAGES = 20;
13
14   /** The mailbox's current historyId, used to seed incremental sync. */
15   export function getCurrentHistoryId(): string | null {
16       const profile = Gmail.Users.getProfile("me");
17       return profile.historyId ?? null;
18   }
19
20   /** What an incremental scan yields: new message ids and the id to store next. */
21   export interface IncrementalChanges {
22       messageIds: string[];
23       historyId: string;
24   }
25
26   /**
27    * Lists messages added since `startHistoryId` via the Gmail advanced service,
28    * paging through results. Returns null when the history has expired (the caller
29    * must fall back to a full sync). Only `messageAdded` events are requested.
30    */
31   export function listAddedSince(

```

```

32     startHistoryId: string,
33 ): IncrementalChanges | null {
34     const responses: HistoryListResponse[] = [];
35     let pageToken: string | undefined;
36     try {
37         for (let page = 0; page < MAX_HISTORY_PAGES; page++) {
38             const response = Gmail.Users.History.list("me", {
39                 startHistoryId,
40                 historyTypes: ["messageAdded"],
41                 pageToken,
42             }) as HistoryListResponse;
43             responses.push(response);
44             if (!response.nextPageToken) {
45                 break;
46             }
47             pageToken = response.nextPageToken;
48         }
49     } catch (e) {
50         if (isHistoryExpired(e)) {
51             logDebug("History id expired; full sync required.");
52             return null;
53         }
54         throw e;
55     }
56     return {
57         messageIds: collectAddedMessageIds(responses),
58         historyId: nextHistoryId(responses, startHistoryId),
59     };
60 }
61
62 /**
63  * Resolves the distinct Gmail threads containing the given message ids. Threads
64  * are de-duplicated by id so a multi-message thread is processed once. Messages
65  * that can no longer be fetched (deleted since the change was recorded) are
66  * skipped.
67  */
68 export function threadsForMessageIds(messageIds: string[]): GmailThread[] {
69     const threads = new Map<string, GmailThread>();
70     for (const id of messageIds) {
71         const message = GmailApp.getMessageById(id);
72         if (!message) {
73             continue;
74         }
75         const thread = message.getThread();
76         threads.set(thread.getId(), thread);
77     }
78     return Array.from(threads.values());
79 }
80

```

9. user (2026-06-10T16:13:48.392Z)

```

1  /**
2   * Pure helpers for the Gmail `users.history.list` incremental sync. These
3   * operate on plain data shapes (the JSON the Gmail advanced service returns)
4   * and have no GAS dependencies, so they are exercised directly in unit tests.
5   * The actual advanced-service calls live in the `gmailHistory` shell.
6   *
7   * See https://developers.google.com/workspace/gmail/api/guides/sync ? a stored
8   * `startHistoryId` returns only the changes since that point (~2 quota units vs
9   * ~5 for a list call). The id can expire, in which case the API responds 404
10  * and the caller must fall back to a full (date-window) sync.
11  */
12
13  /** A message stub as it appears inside a history record. */

```

```

14 export interface HistoryMessage {
15     id?: string;
16     threadId?: string;
17 }
18
19 /** One change record; we only care about messages added to the mailbox. */
20 export interface HistoryRecord {
21     messagesAdded?: { message?: HistoryMessage }[];
22 }
23
24 /** The shape of a `Gmail.Users.History.list` response we depend on. */
25 export interface HistoryListResponse {
26     history?: HistoryRecord[];
27     historyId?: string;
28     nextPageToken?: string;
29 }
30
31 /**
32  * Collects the unique ids of messages added across the given history pages, in
33  * first-seen order. Records without a `messagesAdded` entry (e.g. label-only
34  * changes) contribute nothing.
35  */
36 export function collectAddedMessageIds(
37     responses: HistoryListResponse[],
38 ): string[] {
39     const seen = new Set<string>();
40     const ids: string[] = [];
41     for (const response of responses) {
42         for (const record of response.history ?? []) {
43             for (const added of record.messagesAdded ?? []) {
44                 const id = added.message?.id;
45                 if (id && !seen.has(id)) {
46                     seen.add(id);
47                     ids.push(id);
48                 }
49             }
50         }
51     }
52     return ids;
53 }
54
55 /**
56  * The newest `historyId` to persist for the next run: the last page's id, or
57  * the supplied fallback (the start id) if no page reported one.
58  */
59 export function nextHistoryId(
60     responses: HistoryListResponse[],
61     fallback: string,
62 ): string {
63     for (let i = responses.length - 1; i >= 0; i--) {
64         const id = responses[i].historyId;
65         if (id) {
66             return id;
67         }
68     }
69     return fallback;
70 }
71
72 /**
73  * Whether an error thrown by `history.list` means the stored `startHistoryId`
74  * has expired. Gmail responds with HTTP 404 ("Requested entity was not found")
75  * once the id predates the mailbox's retained history; the caller must then do
76  * a full sync. We match defensively on both the status code and the message
77  * text since the advanced service surfaces errors as a stringified exception.
78  */

```

```

79 export function isHistoryExpired(error: unknown): boolean {
80     const e = error as { message?: string; details?: { code?: number } } | null;
81     if (e && e.details && e.details.code === 404) {
82         return true;
83     }
84     const text = String(e?.message ?? error ?? "").toLowerCase();
85     return text.includes("404") || text.includes("not found");
86 }
87

```

10. user (2026-06-10T16:13:48.835Z)

```

1     import { ENABLE_DEBUGGING } from "./config";
2
3     /**
4      * Minimal structured logger. Each line is prefixed with a level so the Apps
5      * Script execution log / Stackdriver can be filtered (e.g. `severity:ERROR`).
6      * Kept dependency-free and side-effect-only so callers stay simple.
7      */
8     function emit(level: "DEBUG" | "INFO" | "ERROR", data: unknown): void {
9         Logger.log "[" + level + " " + String(data));
10    }
11
12    /** Verbose log, only emitted when ENABLE_DEBUGGING is on. */
13    export function logDebug(data: unknown): void {
14        if (ENABLE_DEBUGGING) {
15            emit("DEBUG", data);
16        }
17    }
18
19    /** Informational log; always emitted. */
20    export function logInfo(data: unknown): void {
21        emit("INFO", data);
22    }
23
24    /** Error log; always emitted and surfaces in Stackdriver as an error. */
25    export function logError(data: unknown): void {
26        emit("ERROR", data);
27    }
28

```

11. user (2026-06-10T16:13:49.079Z)

```

1     /**
2      * Pure naming helpers. None of these touch a Google service, so they are
3      * unit-tested directly without any mocks.
4      */
5
6     /** The folder-organisation strategies the setup form offers. */
7     export const FOLDER_STRUCTURES = [
8         "byContentType",
9         "byMonthly",
10        "byYear",
11    ] as const;
12    export type FolderStructure = (typeof FOLDER_STRUCTURES)[number];
13
14    /** Type guard for an untrusted structure string (e.g. from the setup form). */
15    export function isFolderStructure(value: string): value is FolderStructure {
16        return (FOLDER_STRUCTURES as readonly string[]).includes(value);
17    }
18
19    /** Short month names, indexed by `Date#getMonth()` (0 = January). */
20    const MONTHS = [
21        "Jan",
22        "Feb",

```

```

23     "Mar",
24     "Apr",
25     "May",
26     "Jun",
27     "Jul",
28     "Aug",
29     "Sep",
30     "Oct",
31     "Nov",
32     "Dec",
33 ];
34
35 const pad2 = (n: number): string => String(n).padStart(2, "0");
36
37 /** Characters illegal in Windows file names (also awkward for Drive desktop). */
38 const ILLEGAL_NAME_CHARS = /[<>:"/\|?*]/g;
39
40 /**
41  * Removes characters that are illegal in Windows file names or awkward for the
42  * Google Drive desktop client (which we recommend), and collapses whitespace.
43  *
44  * This matters because attachment names are built from the sender (typically
45  * `Name <a@b.com>`) and the original file name, both of which routinely contain
46  * these characters; unsanitised names fail to sync on Windows. Hyphens are kept
47  * (e.g. `2024-01-statement.pdf` stays intact).
48  */
49 export function sanitizeForDriveName(raw: string): string {
50     return raw.replace(ILLEGAL_NAME_CHARS, " ").replace(/\s+/g, " ").trim();
51 }
52
53 /**
54  * Builds a stable, descriptive, portable file name from the sender, the date
55  * the message was sent and the original attachment name, e.g.
56  * "Some Sender a@b.com-06Jun2012-09-05-03-report.pdf"
57  * The time uses `:` separators (not `.`) and the month is a fixed abbreviation,
58  * so the result is deterministic and valid on every OS.
59  */
60 export function getAttachmentName(
61     fileName: string,
62     from: string,
63     date: Date,
64 ): string {
65     const datePart =
66         pad2(date.getDate()) + MONTHS[date.getMonth()] + date.getFullYear();
67     const timePart =
68         pad2(date.getHours()) +
69         "-" +
70         pad2(date.getMinutes()) +
71         "-" +
72         pad2(date.getSeconds());
73     return (
74         sanitizeForDriveName(from) +
75         "-" +
76         datePart +
77         "-" +
78         timePart +
79         "-" +
80         sanitizeForDriveName(fileName)
81     );
82 }
83
84 /** Folder name like "Jun2012" for the month/year of the given date. */
85 export function getFolderNameForMonthYear(date: Date): string {
86     return MONTHS[date.getMonth()] + date.getFullYear();
87 }

```

```

88
89  /**
90   * Maps a MIME type to a human-friendly bucket folder. Returns "" for content
91   * we do not recognise (the caller treats "" as "no folder").
92   */
93 export function getFolderNameForContentType(type: string): string {
94     switch (type.split("/")[0]) {
95         case "text":
96             return "Text Data";
97         case "image":
98             return "Images";
99         case "application":
100             return "Application Data";
101         case "audio":
102             return "Audio";
103         case "video":
104             return "Video";
105         default:
106             return "";
107     }
108 }
109
110 /**
111  * Resolves the sub-folder name for an attachment given the user's chosen
112  * structure. Returns "" when the structure is unknown.
113  */
114 export function getFolderNameForStructure(
115     structure: string,
116     contentType: string,
117     date: Date,
118 ): string {
119     switch (structure) {
120         case "byContentType":
121             return getFolderNameForContentType(contentType);
122         case "byMonthly":
123             return getFolderNameForMonthYear(date);
124         case "byYear":
125             return String(date.getFullYear());
126         default:
127             return "";
128     }
129 }
130

```

12. user (2026-06-10T16:14:13.374Z)

```

1  import {
2      ENABLE_SUMMARY_EMAIL,
3      SUMMARY_EMAIL_RECIPIENT,
4  } from "./config";
5  import { logError, logInfo } from "./logger";
6  import { dayKey, formatSummary } from "./stats";
7  import * as state from "./state";
8
9  /**
10   * Summarises the previous day's activity the first time a sync runs on a new
11   * calendar day. Always logs the summary; emailing is opt-in
12   * (ENABLE_SUMMARY_EMAIL, which requires the script.send_mail scope).
13   *
14   * `now` is injected for testability.
15   */
16 export function maybeSendDailySummary(now: Date): void {
17     const today = dayKey(now);
18     if (state.getSummaryLastSent() === today) {
19         return; // already summarised since the last day rolled over

```

```

20     }
21
22     // Summarise whatever the persisted accumulator holds (the prior day's run).
23     const stats = state.getRunStatsRaw();
24     if (!stats || stats.day === today) {
25         // Nothing from a prior day to report yet; arm for the next rollover.
26         state.setSummaryLastSent(today);
27         return;
28     }
29
30     const summary = formatSummary(stats);
31     logInfo(summary);
32
33     if (ENABLE_SUMMARY_EMAIL) {
34         try {
35             const to = SUMMARY_EMAIL_RECIPIENT || Session.getEffectiveUser().getEmail();
36             if (to) {
37                 MailApp.sendEmail(to, "Gmail?Drive sync summary " + stats.day, summary);
38             }
39         } catch (e) {
40             logError("Failed to send summary email: " + e);
41         }
42     }
43
44     state.setSummaryLastSent(today);
45 }
46

```

13. user (2026-06-10T16:14:14.319Z)

```

1  /**
2   * Pure helpers for the set of already-processed Gmail message IDs. Keeping this
3   * logic free of any Google service makes it directly unit-testable.
4   *
5   * The set is bounded: PropertiesService caps a single value at 9 KB, and a
6   * message ID is ~16 chars, so we keep the most recent MAX_PROCESSED_IDS ids in
7   * insertion order (oldest dropped first). Re-scanning an old window may then
8   * re-probe a long-evicted message, which is harmless ? the Drive
9   * `getFilesByName` check still prevents a duplicate copy.
10  */
11
12  /** Upper bound on retained ids; ~16 chars each keeps us well under the 9 KB cap. */
13  export const MAX_PROCESSED_IDS = 400;
14
15  /** Parses the persisted JSON array, tolerating null/garbage as "empty". */
16  export function parseProcessedIds(raw: string | null): string[] {
17      if (!raw) {
18          return [];
19      }
20      try {
21          const parsed = JSON.parse(raw);
22          return Array.isArray(parsed) ? (parsed as string[]) : [];
23      } catch {
24          return [];
25      }
26  }
27
28  /**
29   * Returns a new list with `id` appended (if not already present), trimmed to
30   * the newest `max` entries. Existing ids are not reordered.
31   */
32  export function addProcessedId(
33      ids: string[],
34      id: string,
35      max: number = MAX_PROCESSED_IDS,

```

```

36     ): string[] {
37         if (ids.includes(id)) {
38             return ids;
39         }
40         const next = ids.concat(id);
41         return next.length > max ? next.slice(next.length - max) : next;
42     }
43

```

14. user (2026-06-10T16:14:14.783Z)

```

1     import { ONE_DAY, THIRTY_DAYS } from "./config";
2
3     export interface SearchQuery {
4         /** The Gmail search expression. */
5         query: string;
6         /** ISO timestamp of the upper bound of this window (for debugging). */
7         before: string;
8         /** True once the window has reached "now" ? i.e. we are fully caught up. */
9         isToday: boolean;
10    }
11
12    /**
13     * Builds the Gmail search for the next 30-day window after `lastSynced`.
14     *
15     * `now` is injected (rather than read from the clock) so the windowing logic is
16     * deterministic under test. NOTE: this intentionally preserves the legacy
17     * behaviour of mixing a UTC upper bound (`Date.parse`) with local-time day
18     * parts (`getFullYear`/`getMonth`/`getDate`) when formatting the Gmail
19     * `before:`/`after:` strings, so the day boundaries follow the script's
20     * configured timezone.
21     */
22    export function buildSearchQuery(lastSynced: string, now: Date): SearchQuery {
23        let query = "to:me in:all has:attachment -in:trash -in:spam ";
24        let nextDate = THIRTY_DAYS + Date.parse(lastSynced);
25        const today = now.getTime();
26        let isToday = false;
27        if (today < nextDate) {
28            isToday = true;
29            nextDate = today;
30        }
31        nextDate += ONE_DAY;
32
33        const afterDate = new Date(Date.parse(lastSynced) - ONE_DAY);
34        const beforeDate = new Date(nextDate);
35        const beforeQ = `${beforeDate.getFullYear()}/${beforeDate.getMonth() +
1}/${beforeDate.getDate()}`;
36        const afterQ = `${afterDate.getFullYear()}/${afterDate.getMonth() +
1}/${afterDate.getDate()}`;
37
38        if (beforeQ === afterQ) {
39            // Window collapses to a single day; just search "after yesterday".
40            const yesterday = new Date(Date.parse(lastSynced) - ONE_DAY);
41            const yesterdayQ = `${yesterday.getFullYear()}/${yesterday.getMonth() +
1}/${yesterday.getDate()}`;
42            query = query + " " + "after:" + yesterdayQ;
43        } else {
44            query = query + "before:" + beforeQ + " " + "after:" + afterQ;
45        }
46
47        return { query, before: beforeDate.toJSON(), isToday };
48    }
49

```

15. user (2026-06-10T16:14:14.965Z)


```

1  /** Bookkeeping for how many files we have added on a given calendar day. */
2  export interface FilesAddedToday {
3      added: number;
4      /** ISO timestamp of the day this count belongs to. */
5      date: string;
6  }
7
8  /**
9   * True while we are still within our wall-clock budget for a single execution
10   * (Apps Script kills long-running scripts). `now` and `budgetMs` are injected
11   * for testability.
12   */
13  export function hasTime(now: Date, start: Date, budgetMs: number): boolean {
14      return now.getTime() - start.getTime() < budgetMs;
15  }
16
17  /**
18   * Whether two dates fall on the same calendar day. Unlike the legacy code
19   * (which compared only day-of-month), this also compares month and year, so
20   * the daily counter resets correctly across month boundaries.
21   */
22  export function isSameCalendarDay(a: Date, b: Date): boolean {
23      return (
24          a.getFullYear() === b.getFullYear() &&
25          a.getMonth() === b.getMonth() &&
26          a.getDate() === b.getDate()
27      );
28  }
29

```

16. user (2026-06-10T16:14:21.658Z)

```

1  import {
2      PROP_DEBUG,
3      PROP_FILES_ADDED_TODAY,
4      PROP_FOLDER,
5      PROP_HISTORY_ID,
6      PROP_LAST_ERROR,
7      PROP_LAST_SYNCED,
8      PROP_PROCESSED_IDS,
9      PROP_RUN_STATS,
10     PROP_SUMMARY_LAST_SENT,
11 } from "./config";
12 import { parseProcessedIds } from "./processed";
13 import { parseStats, parseStatsRaw, type RunStats } from "./stats";
14 import type { FilesAddedToday } from "./quota";
15
16  /**
17   * Thin wrappers over PropertiesService. All persistent sync state lives in the
18   * user properties store. Replaces the removed `UserProperties` global and the
19   * deprecated `Utilities.jsonParse/jsonStringify` helpers.
20   */
21  function props(): GoogleAppsScript.Properties.Properties {
22      return PropertiesService.getUserProperties();
23  }
24
25  export function getFolderStructure(): string {
26      return props().getProperty(PROP_FOLDER) ?? "";
27  }
28
29  export function setFolderStructure(structure: string): void {
30      props().setProperty(PROP_FOLDER, structure);
31  }
32
33  export function getLastSynced(): string {

```

```

34     return props().getProperty(PROP_LAST_SYNCED) ?? "";
35 }
36
37 export function setLastSynced(iso: string): void {
38     props().setProperty(PROP_LAST_SYNCED, iso);
39 }
40
41 export function getDebug(): string | null {
42     return props().getProperty(PROP_DEBUG);
43 }
44
45 export function setDebug(value: string): void {
46     props().setProperty(PROP_DEBUG, value);
47 }
48
49 export function getFilesAddedToday(): FilesAddedToday {
50     const raw = props().getProperty(PROP_FILES_ADDED_TODAY);
51     if (!raw) {
52         return { added: 0, date: new Date().toJSON() };
53     }
54     return JSON.parse(raw) as FilesAddedToday;
55 }
56
57 export function setFilesAddedToday(value: FilesAddedToday): void {
58     props().setProperty(PROP_FILES_ADDED_TODAY, JSON.stringify(value));
59 }
60
61 /** The set of Gmail message ids we have already processed. */
62 export function getProcessedIds(): string[] {
63     return parseProcessedIds(props().getProperty(PROP_PROCESSED_IDS));
64 }
65
66 export function setProcessedIds(ids: string[]): void {
67     props().setProperty(PROP_PROCESSED_IDS, JSON.stringify(ids));
68 }
69
70 /** Per-day run statistics accumulator (fresh for `day` if absent/stale). */
71 export function getRunStats(day: string): RunStats {
72     return parseStats(props().getProperty(PROP_RUN_STATS), day);
73 }
74
75 /** The persisted run stats as-is (may belong to a prior day), or null. */
76 export function getRunStatsRaw(): RunStats | null {
77     return parseStatsRaw(props().getProperty(PROP_RUN_STATS));
78 }
79
80 export function setRunStats(stats: RunStats): void {
81     props().setProperty(PROP_RUN_STATS, JSON.stringify(stats));
82 }
83
84 /** The day key (YYYY-MM-DD) of the last summary we sent, or null. */
85 export function getSummaryLastSent(): string | null {
86     return props().getProperty(PROP_SUMMARY_LAST_SENT);
87 }
88
89 export function setSummaryLastSent(day: string): void {
90     props().setProperty(PROP_SUMMARY_LAST_SENT, day);
91 }
92
93 /**
94  * The Gmail `historyId` from which incremental sync should resume, or null when
95  * we are still backfilling (date-window mode) or the id has expired.
96  */
97 export function getHistoryId(): string | null {
98     return props().getProperty(PROP_HISTORY_ID);

```

```

99     }
100
101     export function setHistoryId(id: string): void {
102         props().setProperty(PROP_HISTORY_ID, id);
103     }
104
105     /** Drops the stored historyId, forcing the next run back into backfill mode. */
106     export function clearHistoryId(): void {
107         props().deleteProperty(PROP_HISTORY_ID);
108     }
109
110     /** Records the last execution error so it can be inspected later. */
111     export function setLastError(message: string): void {
112         props().setProperty(
113             PROP_LAST_ERROR,
114             JSON.stringify({ message, at: new Date().toJSON() }),
115         );
116     }
117
118     /** Wipes all persisted state. Drive files are unaffected. */
119     export function clearAll(): void {
120         props().deleteAllProperties();
121     }
122

```

17. user (2026-06-10T16:14:22.025Z)

```

1     /**
2      * Pure helpers for per-day run statistics. The accumulator is persisted as JSON
3      * in user properties; keeping the math here (no Google services) makes it
4      * directly unit-testable.
5      */
6
7     export interface RunStats {
8         /** Calendar day these stats belong to (YYYY-MM-DD, local time). */
9         day: string;
10        /** Number of sync passes that ran on this day. */
11        runs: number;
12        /** Files copied to Drive on this day. */
13        filesAdded: number;
14        /** Attachment/message errors encountered on this day. */
15        errors: number;
16        /** Most recent error message of the day, if any. */
17        lastError: string;
18    }
19
20    /** Local-time YYYY-MM-DD key for a date. */
21    export function dayKey(date: Date): string {
22        const y = date.getFullYear();
23        const m = String(date.getMonth() + 1).padStart(2, "0");
24        const d = String(date.getDate()).padStart(2, "0");
25        return `${y}-${m}-${d}`;
26    }
27
28    export function emptyStats(day: string): RunStats {
29        return { day, runs: 0, filesAdded: 0, errors: 0, lastError: "" };
30    }
31
32    /** Parses persisted stats as-is, or null if absent/malformed. Day is not coerced. */
33    export function parseStatsRaw(raw: string | null): RunStats | null {
34        if (!raw) {
35            return null;
36        }
37        try {
38            const parsed = JSON.parse(raw) as Partial<RunStats>;

```

```

39     if (!parsed || typeof parsed.day !== "string") {
40         return null;
41     }
42     return {
43         day: parsed.day,
44         runs: parsed.runs ?? 0,
45         filesAdded: parsed.filesAdded ?? 0,
46         errors: parsed.errors ?? 0,
47         lastError: parsed.lastError ?? "",
48     };
49 } catch {
50     return null;
51 }
52 }
53
54 /** Parses persisted stats, returning a fresh accumulator for `day` if absent/stale. */
55 export function parseStats(raw: string | null, day: string): RunStats {
56     const parsed = parseStatsRaw(raw);
57     return parsed && parsed.day === day ? parsed : emptyStats(day);
58 }
59
60 export interface RunOutcome {
61     filesAdded: number;
62     errors: number;
63     lastError?: string;
64 }
65
66 /** Folds a single run's outcome into the day's accumulator (returns a new object). */
67 export function recordRun(stats: RunStats, outcome: RunOutcome): RunStats {
68     return {
69         day: stats.day,
70         runs: stats.runs + 1,
71         filesAdded: stats.filesAdded + outcome.filesAdded,
72         errors: stats.errors + outcome.errors,
73         lastError: outcome.lastError ? outcome.lastError : stats.lastError,
74     };
75 }
76
77 /** Human-readable one-screen summary used for logs and the optional email. */
78 export function formatSummary(stats: RunStats): string {
79     const lines = [
80         `Gmail?Drive sync summary for ${stats.day}`,
81         `  runs:          ${stats.runs}`,
82         `  files added:  ${stats.filesAdded}`,
83         `  errors:       ${stats.errors}`,
84     ];
85     if (stats.lastError) {
86         lines.push(`  last error: ${stats.lastError}`);
87     }
88     return lines.join("\n");
89 }
90

```

18. user (2026-06-10T16:14:29.041Z)

```

1  import {
2      ENABLE_INCREMENTAL_SYNC,
3      MAX_FILES_ADD_PER_DAY,
4      MAX_RUNTIME_MS,
5      MAX_SYNC_TRIGGERS,
6      ONE_DAY,
7      ONE_MINUTE,
8      SIX_HOURS,
9      THIRTY_DAYS,
10 } from "./config";

```

```

11 import { updateDrive, type UpdateResult } from "../drive";
12 import {
13     getCurrentHistoryId,
14     listAddedSince,
15     threadsForMessageIds,
16 } from "../gmailHistory";
17 import { logDebug, logError } from "../logger";
18 import { maybeSendDailySummary } from "../notify";
19 import { addProcessedId } from "../processed";
20 import { buildSearchQuery } from "../query";
21 import { hasTime, isSameCalendarDay } from "../quota";
22 import { dayKey, recordRun } from "../stats";
23 import * as state from "../state";
24
25 const SYNC_HANDLER = "syncUserDrives";
26
27 /** Schedules a one-shot run of syncUserDrives `afterMs` from now. */
28 export function newSyncTrigger(afterMs: number): void {
29     ScriptApp.newTrigger(SYNC_HANDLER).timeBased().after(afterMs).create();
30 }
31
32 /** All currently-installed triggers for the sync handler. */
33 function getSyncTriggers(): GoogleAppsScript.Script.Trigger[] {
34     return ScriptApp.getProjectTriggers().filter(
35         (t) => t.getHandlerFunction() === SYNC_HANDLER,
36     );
37 }
38
39 /** Removes every project trigger (used when the user re-runs setup). */
40 export function deleteAllTriggers(): void {
41     for (const trigger of ScriptApp.getProjectTriggers()) {
42         ScriptApp.deleteTrigger(trigger);
43     }
44 }
45
46 function deleteTriggers(triggers: GoogleAppsScript.Script.Trigger[]): void {
47     for (const trigger of triggers) {
48         ScriptApp.deleteTrigger(trigger);
49     }
50 }
51
52 /** Defense-in-depth: never let sync triggers exceed the cap (limit is 20). */
53 function pruneSyncTriggers(max: number): void {
54     const triggers = getSyncTriggers();
55     for (let i = max; i < triggers.length; i++) {
56         ScriptApp.deleteTrigger(triggers[i]);
57     }
58 }
59
60 /**
61  * The background worker. A per-user lock ensures only one run executes at a time
62  * for THIS user, so concurrent triggers cannot stack file copies or trigger
63  * generations. It is deliberately a user lock, not a script lock: the web app
64  * runs as the accessing user (executeAs USER_ACCESSING), so a script-wide lock
65  * would serialise every user's sync into one global queue.
66  */
67 export function syncUserDrives(): void {
68     const lock = LockService.getUserLock();
69     if (!lock.tryLock(0)) {
70         logDebug("Another sync run is in progress; skipping.");
71         return;
72     }
73     try {
74         runSyncOnce();
75     } finally {

```

```

76         // Bound the trigger count regardless of which path we took, then release.
77         pruneSyncTriggers(MAX_SYNC_TRIGGERS);
78         lock.releaseLock();
79     }
80 }
81
82 /**
83  * A single sync pass:
84  *   1. reschedules a fan-out of backup triggers (self-healing if one is lost),
85  *   2. syncs forward in 30-day windows until it runs out of time, hits the
86  *       daily file cap, or fully catches up,
87  *   3. persists progress and schedules the next run.
88  */
89 function runSyncOnce(): void {
90     const today = new Date();
91
92     // On the first run of a new calendar day, summarise the previous day.
93     maybeSendDailySummary(today);
94
95     // Recover state if the debug flag was cleared (e.g. properties were wiped).
96     const debug = state.getDebug();
97     if (debug == null || debug === "0") {
98         state.setDebug("1");
99         state.setLastSynced(new Date(today.getTime() - 365 * ONE_DAY).toJSON());
100    }
101
102    // Create the backup triggers BEFORE deleting the current ones, so a failure
103    // here can never leave us with no scheduled run.
104    const previousTriggers = getSyncTriggers();
105    try {
106        newSyncTrigger(60 * ONE_MINUTE);
107        newSyncTrigger(SIX_HOURS);
108        newSyncTrigger(2 * SIX_HOURS);
109        newSyncTrigger(4 * SIX_HOURS);
110        newSyncTrigger(8 * SIX_HOURS);
111        newSyncTrigger(28 * SIX_HOURS);
112    } catch (e) {
113        // Could not install new triggers; leave the old ones and try again later.
114        logError("Could not install triggers: " + e);
115        return;
116    }
117    deleteTriggers(previousTriggers);
118
119    const filesAddedToday = state.getFilesAddedToday();
120    let added = filesAddedToday.added;
121    const lastCountDate = new Date(Date.parse(filesAddedToday.date));
122
123    if (isSameCalendarDay(lastCountDate, today) && added >= MAX_FILES_ADD_PER_DAY) {
124        logDebug("Used up all quota for today. Can't add more files.");
125        newSyncTrigger(60 * ONE_MINUTE);
126        return;
127    }
128    if (!isSameCalendarDay(lastCountDate, today)) {
129        added = 0;
130    }
131
132    const structure = state.getFolderStructure();
133    let lastSynced = state.getLastSynced();
134
135    // Load the dedup set once; thread it through the run and persist at the end.
136    let processedIds = state.getProcessedIds();
137    const processedSet = new Set(processedIds);
138
139    // This run's contribution to the day's stats.
140    const addedAtStart = added;

```

```

141     let runErrors = 0;
142     let runLastError = "";
143
144     // Folds one updateDrive result into this run's accumulators.
145     const applyResult = (result: UpdateResult): void => {
146         added += result.addedFiles;
147         runErrors += result.errors;
148         if (result.lastError) {
149             runLastError = result.lastError;
150         }
151         for (const id of result.processedIds) {
152             if (!processedSet.has(id)) {
153                 processedSet.add(id);
154                 processedIds = addProcessedId(processedIds, id);
155             }
156         }
157     };
158
159     try {
160         // Incremental mode is opt-in (ENABLE_INCREMENTAL_SYNC) so it rolls out
161         // gradually over the proven date-window backfill. A stored historyId only
162         // exists once a backfill has caught up while the flag was on.
163         const historyId = ENABLE_INCREMENTAL_SYNC ? state.getHistoryId() : null;
164         if (historyId) {
165             // Incremental: process only messages added since the stored historyId.
166             const changes = listAddedSince(historyId);
167             if (changes == null) {
168                 // History expired: clear the cursor and re-scan a bounded recent window
169                 // (the next catch-up re-captures a fresh historyId). Overlap is safe ?
170                 // dedup by message id and the Drive filename guard prevent re-copying.
171                 state.clearHistoryId();
172                 lastSynced = new Date(today.getTime() - THIRTY_DAYS).toJSON();
173                 logDebug("History expired; reverting to date-window sync.");
174             } else {
175                 if (changes.messageIds.length > 0 && added < MAX_FILES_ADD_PER_DAY) {
176                     const threads = threadsForMessageIds(changes.messageIds);
177                     if (threads.length > 0) {
178                         applyResult(
179                             updateDrive(
180                                 threads,
181                                 structure,
182                                 MAX_FILES_ADD_PER_DAY - added,
183                                 processedSet,
184                             ),
185                         );
186                     }
187                 }
188                 // Advance the cursor so the next run only sees newer changes.
189                 state.setHistoryId(changes.historyId);
190             }
191         } else {
192             // Backfill: walk forward in 30-day windows until caught up to today.
193             // Capture the mailbox historyId BEFORE processing so mail arriving
194             // mid-run is still picked up by the first incremental scan.
195             const candidateHistoryId = ENABLE_INCREMENTAL_SYNC
196                 ? getCurrentHistoryId()
197                 : null;
198             let fullSynced = false;
199             while (
200                 hasTime(new Date(), today, MAX_RUNTIME_MS) &&
201                 added < MAX_FILES_ADD_PER_DAY &&
202                 !fullSynced
203             ) {
204                 const maxToAdd = MAX_FILES_ADD_PER_DAY - added;
205                 const search = buildSearchQuery(lastSynced, new Date());

```

```

206         logDebug("Query: " + search.query + " files asking: " + maxToAdd);
207
208         const threads = GmailApp.search(search.query);
209         if (threads.length > 0) {
210             applyResult(updateDrive(threads, structure, maxToAdd, processedSet));
211         }
212
213         fullSynced = search.isToday;
214         if (added < MAX_FILES_ADD_PER_DAY) {
215             lastSynced = fullSynced
216                 ? today.toJSON()
217                 : new Date(Date.parse(lastSynced) + THIRTY_DAYS).toJSON();
218         }
219     }
220     // Caught up: switch future runs to the cheaper incremental path.
221     if (candidateHistoryId && fullSynced && added < MAX_FILES_ADD_PER_DAY) {
222         state.setHistoryId(candidateHistoryId);
223         logDebug("Backfill complete; stored historyId " + candidateHistoryId);
224     }
225 }
226 } catch (e) {
227     logError("Some error in execution: " + e);
228     state.setLastError(String(e));
229     runErrors++;
230     runLastError = String(e);
231 }
232
233 // Fold this run into the day's accumulator.
234 const dayStats = recordRun(state.getRunStats(dayKey(today)), {
235     filesAdded: added - addedAtStart,
236     errors: runErrors,
237     lastError: runLastError,
238 });
239 state.setRunStats(dayStats);
240
241 state.setProcessedIds(processedIds);
242 state.setLastSynced(lastSynced);
243 state.setFilesAddedToday({ added, date: today.toJSON() });
244 newSyncTrigger(20 * ONE_MINUTE);
245 }
246

```

19. user (2026-06-10T16:14:36.887Z)

```

1     import {
2         MAX_FILES_ADD_PER_DAY,
3         ONE_MINUTE,
4         ONE_YEAR,
5         PRIVACY_POLICY_URL,
6         SUPPORT_URL,
7     } from "./config";
8     import { logError } from "./logger";
9     import { isFolderStructure } from "./naming";
10    import * as state from "./state";
11    import { deleteAllTriggers, newSyncTrigger } from "./sync";
12
13    /** Bounds on the "how many years to sync" choice offered by the form. */
14    const MIN_SYNC_YEARS = 1;
15    const MAX_SYNC_YEARS = 10;
16
17    /** Web-app entry point: serves the setup form. */
18    export function doGet(): GoogleAppsScript.HTML.HtmlOutput {
19        return HtmlService.createHtmlOutputFromFile("index");
20    }
21

```



```

22  /** Shape of the form submitted from index.html. */
23  export interface SetupForm {
24      folderStructure: string;
25      syncFrom: string;
26  }
27
28  /**
29   * Called from the setup form. Resets this user's state (their Drive files are
30   * untouched), stores their preferences, and kicks off the first background
31   * sync. The web app runs as the accessing user (executeAs USER_ACCESSING), so
32   * everything here operates on the current user's own Gmail/Drive/properties.
33   *
34   * Wrapped so a permission problem (e.g. the user declined a scope on the
35   * granular consent screen) returns a clear message instead of a raw stack
36   * trace. Without all scopes granted the background trigger would otherwise fail
37   * silently later.
38   */
39  export function processInput(form: SetupForm): string {
40      // Validate the untrusted form before touching any state. An unknown structure
41      // would otherwise dump every file into the Drive root (with a logged error per
42      // attachment), and a non-numeric "years" would corrupt lastSynced.
43      const structure = form?.folderStructure ?? "";
44      const syncYears = Number.parseInt(form?.syncFrom ?? "", 10);
45      if (!isFolderStructure(structure)) {
46          return buildValidationErrorMessage("Please choose a folder structure.");
47      }
48      if (
49          !Number.isInteger(syncYears) ||
50          syncYears < MIN_SYNC_YEARS ||
51          syncYears > MAX_SYNC_YEARS
52      ) {
53          return buildValidationErrorMessage(
54              "Please choose how many years of attachments to sync.",
55          );
56      }
57
58      try {
59          // The user is (re)running setup; flush their past state and triggers.
60          state.clearAll();
61          deleteAllTriggers();
62
63          state.setFolderStructure(structure);
64
65          const startMillis = Date.now() - syncYears * ONE_YEAR;
66          state.setLastSynced(new Date(startMillis).toJSON());
67          state.setDebug("1");
68          state.setFilesAddedToday({ added: 0, date: new Date().toJSON() });
69
70          // One-time trigger so we return to the user immediately and sync in the
71          // background.
72          newSyncTrigger(2 * ONE_MINUTE);
73
74          return buildSuccessMessage();
75      } catch (e) {
76          logError("Setup failed: " + e);
77          return buildAuthErrorMessage();
78      }
79  }
80
81  function buildValidationErrorMessage(detail: string): string {
82      return (
83          "<br/><br/><br/><b>" +
84          detail +
85          "</b><br/><br/>Please go back, complete the form, and submit again."
86      );

```

```

87     }
88
89     function buildSuccessMessage(): string {
90         return (
91             "<br/><br/>You're all set ? Gmail Attachments to Drive will start " +
92             "copying your attachments into a <b>GmailAttachments</b> folder in " +
93             "<i>your</i> Google Drive within a minute." +
94             "<br/><br/>Google Apps Script limits us to " +
95             MAX_FILES_ADD_PER_DAY +
96             " files per day and caps daily processing time, so the first sync may take " +
97             "a couple of days to catch up. After that, new attachments usually appear " +
98             "within 5?10 minutes." +
99             "<br/><br/>Tip: install Google Drive for desktop and link this account, so " +
100            "these attachments sync to your computer automatically." +
101            "<br/><br/><b>Privacy:</b> the app reads your Gmail attachments and copies " +
102            "them into your own Drive. Nothing leaves your Google account. " +
103            '<a href="' +
104            PRIVACY_POLICY_URL +
105            '" target="_blank">Privacy policy</a>.' +
106            "<br/><br/>Questions or issues? " +
107            '<a href="' +
108            SUPPORT_URL +
109            '" target="_blank">Project page & support</a>.'
110        );
111    }
112
113    function buildAuthErrorMessage(): string {
114        return (
115            "<br/><br/><b>Setup couldn't finish.</b>" +
116            "<br/><br/>This usually means not all permissions were granted. Please " +
117            "reload this page and, when Google asks, <b>allow all</b> the requested " +
118            "permissions (read your Gmail, manage your Drive files, and run in the " +
119            "background) so the app can sync your attachments." +
120            "<br/><br/>If it keeps failing, see the " +
121            '<a href="' +
122            SUPPORT_URL +
123            '" target="_blank">project page</a>.'
124        );
125    }
126

```

20. user (2026-06-10T16:14:37.683Z)

```

1    /*
2     * Minimal ambient declarations for the parts of the Gmail advanced service we
3     * use. `@types/google-apps-script` types the built-in `GmailApp` but not the
4     * advanced `Gmail` service, which the runtime injects as a global when enabled
5     * in appsscript.json. We declare only what `gmailHistory.ts` calls.
6     */
7    declare namespace GmailAdvanced {
8        interface Profile {
9            emailAddress?: string;
10           historyId?: string;
11        }
12
13        interface HistoryMessageRef {
14            id?: string;
15            threadId?: string;
16        }
17
18        interface HistoryItem {
19            id?: string;
20            messagesAdded?: { message?: HistoryMessageRef }[];
21        }
22

```

```

23     interface HistoryList {
24         history?: HistoryItem[];
25         historyId?: string;
26         nextPageToken?: string;
27     }
28
29     interface HistoryListOptions {
30         startHistoryId: string;
31         historyTypes?: string[];
32         maxResults?: number;
33         pageToken?: string;
34     }
35
36     interface UsersCollection {
37         getProfile(userId: string): Profile;
38         History: {
39             list(userId: string, optionalArgs?: HistoryListOptions): HistoryList;
40         };
41     }
42
43     interface GmailService {
44         Users: UsersCollection;
45     }
46 }
47
48 declare const Gmail: GmailAdvanced.GmailService;
49

```

21. user (2026-06-10T16:14:42.734Z)

```

1  {
2      "timeZone": "America/Los_Angeles",
3      "runtimeVersion": "V8",
4      "exceptionLogging": "STACKDRIVER",
5      "dependencies": {
6          "enabledAdvancedServices": [
7              {
8                  "userSymbol": "Gmail",
9                  "serviceId": "gmail",
10                 "version": "v1"
11             }
12         ]
13     },
14     "oauthScopes": [
15         "https://www.googleapis.com/auth/gmail.readonly",
16         "https://www.googleapis.com/auth/drive",
17         "https://www.googleapis.com/auth/script.scriptapp"
18     ],
19     "webapp": {
20         "access": "ANYONE",
21         "executeAs": "USER_ACCESSING"
22     }
23 }
24

```

22. user (2026-06-10T16:14:43.226Z)

```

1  import { build } from "esbuild";
2  import { copyFileSync, mkdirSync } from "node:fs";
3
4  mkdirSync("dist", { recursive: true });
5
6  await build({
7      entryPoints: ["src/entry.ts"],
8      bundle: true,

```

```

9      format: "iife",
10     // Expose the module's exports as a single global the footer delegates into.
11     globalName: "GmailToDrive",
12     // Apps Script runs on V8; es2019 is a safe floor and lets esbuild downlevel
13     // newer syntax (e.g. ??). Whitespace may be trimmed, but identifiers MUST be
14     // preserved so trigger/web-app name lookups keep working.
15     target: "es2019",
16     minifyIdentifiers: false,
17     outfile: "dist/Code.js",
18     // Real top-level function declarations: they resolve by name at runtime AND
19     // show up in the Apps Script editor's Run dropdown.
20     footer: {
21       js: [
22         "function doGet(e) { return GmailToDrive.doGet(e); }",
23         "function processInput(form) { return GmailToDrive.processInput(form); }",
24         "function syncUserDrives(e) { return GmailToDrive.syncUserDrives(e); }",
25       ].join("\n"),
26     },
27   });
28
29   // clasp pushes everything under dist/, so the manifest and HTML live there too.
30   copyFileSync("appsscript.json", "dist/appsscript.json");
31   copyFileSync("html/index.html", "dist/index.html");
32
33   console.log("Build complete -> dist/Code.js");
34

```

23. user (2026-06-10T16:14:43.695Z)

```

1  /** @type {import('ts-jest').JestConfigWithTsJest} */
2  module.exports = {
3    preset: "ts-jest",
4    testEnvironment: "node",
5    setupFiles: ["<rootDir>/tests/setup.ts"],
6    testMatch: ["**/tests/**/*.test.ts"],
7    collectCoverageFrom: [
8      "src/**/*.ts",
9      // entry.ts is bundler glue (re-exports only); it has no logic to test.
10     "src/entry.ts",
11     // Ambient type declarations carry no runtime code.
12     "src/**/*.d.ts",
13   ],
14   coverageThreshold: {
15     global: {
16       statements: 85,
17       branches: 75,
18       functions: 85,
19       lines: 85,
20     },
21   },
22 };
23

```

24. user (2026-06-10T16:14:44.175Z)

```

1  name: CI
2
3  on:
4    push:
5    pull_request:
6
7  jobs:
8    build-and-test:
9      runs-on: ubuntu-latest
10     steps:

```

```

11     - uses: actions/checkout@v5
12     - uses: actions/setup-node@v5
13     with:
14         node-version: 24
15         cache: npm
16     - run: npm ci
17     - run: npm run typecheck
18     - run: npm run lint
19     - run: npm run test:coverage
20     - run: npm run build
21
22     deploy:
23       needs: build-and-test
24       if: github.event_name == 'push' && github.ref == 'refs/heads/master'
25       runs-on: ubuntu-latest
26       steps:
27         - uses: actions/checkout@v5
28         - uses: actions/setup-node@v5
29         with:
30             node-version: 24
31             cache: npm
32         - run: npm ci
33         - run: npm run build
34         - name: Push and deploy with clasp
35         env:
36             CLASPRC_JSON: ${ secrets.CLASPRC_JSON }
37         run: |
38             if [ -z "$CLASPRC_JSON" ]; then
39                 echo "::warning::CLASPRC_JSON secret not set; skipping deploy."
40                 exit 0
41             fi
42             # clasp reads OAuth credentials from ~/.clasprc.json; the project link
43             # (scriptId + rootDir) comes from the committed .clasp.json.
44             printf '%s' "$CLASPRC_JSON" > "$HOME/.clasprc.json"
45             # Fail early with an actionable message if the secret is not in the
46             # shape clasp v2 expects (a nested "token" with an access_token).
47             # clasp v3's `clasp login` writes a different layout, which otherwise
48             # surfaces only as the cryptic "Cannot read properties of undefined
49             # (reading 'access_token')". Regenerate with: npx @google/clasp@2 login
50             node -e 'const r=require(process.env.HOME+"/.clasprc.json");if(!r||!r.token||!
r.token.access_token){console.error("::error::CLASPRC_JSON is not in clasp v2 format (missing
token.access_token). Regenerate with: npx @google/clasp@2 login, then copy ~/.clasprc.json into
the CLASPRC_JSON secret.");process.exit(1);}'
51             # Use the project-pinned clasp (v2.5.0); never silently fetch a newer
52             # major from the network, whose CLI/creds format would differ.
53             npx --no-install clasp push --force
54             npx --no-install clasp deploy --description "CI ${GITHUB_SHA}"
55

```

25. user (2026-06-10T16:14:50.165Z)

```

1     # Privacy Policy ? Gmail Attachments to Drive
2
3     _Last updated: 2026-05-31_
4
5     **Gmail Attachments to Drive** ("the app") is a Google Apps Script web app that
6     copies attachments from your Gmail into a `GmailAttachments` folder in your own
7     Google Drive.
8
9     ## What the app accesses
10
11     When you authorize the app, it requests permission to:
12
13     - **Read your Gmail** (`gmail.readonly`) ? to find messages with attachments and
14       read those attachments so they can be copied.

```

```

15 - **Manage your Google Drive** (`drive`) ? to create the `GmailAttachments`
16     folder and its sub-folders and to write the copied attachment files.
17 - **Run in the background** (`script.scriptapp`) ? to schedule the periodic sync
18     on your behalf.
19
20 ## How your data is used
21
22 - The app runs **entirely within your own Google account**, as you
23     (the web app is deployed with `executeAs: USER_ACCESSING`).
24 - Attachments are copied **from your Gmail to your Drive only**. Your email
25     content and files are **never sent to, stored on, or transmitted through any
26     third-party server**. The author of the app has no access to your data.
27 - The only data the app stores is small bookkeeping state (your folder-structure
28     choice, a sync cursor, counters, and recently-processed message IDs) in
29     **Google's per-user `PropertiesService`**, inside your account.
30
31 ## Data sharing and retention
32
33 - **No data is shared** with the developer or any third party.
34 - Bookkeeping state lives in your account for as long as the app is installed.
35     Re-running setup clears it. Files already copied to your Drive are yours and
36     are never modified or deleted by the app.
37
38 ## Revoking access
39
40 You can revoke the app's access at any time at
41 \[myaccount.google.com/permissions\](https://myaccount.google.com/permissions).
42 This stops future syncs; attachments already copied to your Drive remain.
43
44 ## Contact
45
46 Questions or issues:
47 \[github.com/avidullu/gmail-attachments-to-gdrive\](https://github.com/avidullu/gmail-attachments-to-gdrive)
48     (open an issue).
49

```

26. user (2026-06-10T16:14:50.669Z)

```

1  <!--
2  The MIT License (MIT)
3  Copyright (c) 2013 Avi Dullu
4
5  Permission is hereby granted, free of charge, to any person obtaining a copy
6  of this software and associated documentation files (the "Software"), to deal
7  in the Software without restriction, including without limitation the rights
8  to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
9  copies of the Software, and to permit persons to whom the Software is
10     furnished to do so, subject to the following conditions:
11
12     The above copyright notice and this permission notice shall be included in
13     all copies or substantial portions of the Software.
14
15     THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
16     IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
17     FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
18     AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
19     LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
20     OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
21     THE SOFTWARE.
22     -->
23
24  <html>
25      <head>
26          <title>Gmail Attachments to GDrive</title>

```



```

8      "build": "node esbuild.config.mjs",
9      "typecheck": "tsc --noEmit",
10     "lint": "eslint src tests",
11     "test": "jest",
12     "test:coverage": "jest --coverage",
13     "push": "npm run build && clasp push",
14     "deploy": "npm run build && clasp deploy"
15   },
16   "devDependencies": {
17     "@google/clasp": "^2.4.2",
18     "@types/google-apps-script": "^1.0.83",
19     "@types/jest": "^29.5.12",
20     "@types/node": "^20.12.7",
21     "esbuild": "^0.21.5",
22     "eslint": "^9.5.0",
23     "jest": "^29.7.0",
24     "ts-jest": "^29.1.4",
25     "typescript": "^5.4.5",
26     "typescript-eslint": "^8.0.0"
27   }
28 }
29

```

28. assistant (2026-06-10T16:14:57.106Z)

Now let me read the key tests to confirm intent.

29. user (2026-06-10T16:14:57.797Z)

```

1  import {
2    MAX_FILES_ADD_PER_DAY,
3    ONE_DAY,
4    PROP_DEBUG,
5    PROP_FILES_ADDED_TODAY,
6    PROP_FOLDER,
7    PROP_LAST_SYNCED,
8    PROP_PROCESSED_IDS,
9  } from "../src/config";
10 import { syncUserDrives } from "../src/sync";
11
12 type Store = Record<string, string>;
13
14 function fakeProps(initial: Store) {
15   const store: Store = { ...initial };
16   return {
17     store,
18     getProperty: (k: string) => (k in store ? store[k] : null),
19     setProperty: (k: string, v: string) => {
20       store[k] = String(v);
21     },
22     deleteAllProperties: () => {
23       for (const k of Object.keys(store)) delete store[k];
24     },
25   };
26 }
27
28 interface FakeTrigger {
29   getHandlerFunction: () => string;
30 }
31
32 function fakeScriptApp() {
33   const triggers: FakeTrigger[] = [];
34   return {
35     triggers,
36     newTrigger: (handler: string) => ({

```



```

37         timeBased: () => ({
38             after: () => ({
39                 create: () => {
40                     triggers.push({ getHandlerFunction: () => handler });
41                 },
42             })),
43         })),
44     })),
45     getProjectTriggers: () => [...triggers],
46     deleteTrigger: (t: FakeTrigger) => {
47         const i = triggers.indexOf(t);
48         if (i >= 0) triggers.splice(i, 1);
49     },
50 };
51 }
52
53 function fakeLock(canLock: boolean) {
54     const releaseLock = jest.fn();
55     return {
56         releaseLock,
57         service: {
58             getUserLock: () => ({ tryLock: () => canLock, releaseLock }),
59         },
60     };
61 }
62
63 const g = globalThis as unknown as Record<string, unknown>;
64 const nowIso = () => new Date().toJSON();
65
66 function install(props: ReturnType<typeof fakeProps>, lockable = true) {
67     const script = fakeScriptApp();
68     const lock = fakeLock(lockable);
69     g.PropertiesService = { getUserProperties: () => props };
70     g.ScriptApp = script;
71     g.LockService = lock.service;
72     g.GmailApp = { search: jest.fn(() => []) };
73     return { script, lock };
74 }
75
76 function baseProps(overrides: Store = {}) {
77     return fakeProps({
78         [PROP_DEBUG]: "1",
79         [PROP_FOLDER]: "byContentType",
80         [PROP_LAST_SYNCED]: nowIso(),
81         [PROP_FILES_ADDED_TODAY]: JSON.stringify({ added: 0, date: nowIso() }),
82         ...overrides,
83     });
84 }
85
86 describe("syncUserDrives", () => {
87     it("skips entirely when the lock is held by another run", () => {
88         const props = baseProps();
89         const { script } = install(props, /* lockable */ false);
90
91         syncUserDrives();
92
93         expect(script.triggers).toHaveLength(0);
94         expect(g.GmailApp).toBeDefined();
95         expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
96     });
97
98     it("replaces the previous trigger generation and schedules new ones", () => {
99         const props = baseProps();
100         const { script, lock } = install(props);
101         // Seed a stale trigger from a previous run.

```

```

102     const stale = { getHandlerFunction: () => "syncUserDrives" };
103     script.triggers.push(stale);
104
105     syncUserDrives();
106
107     expect(script.triggers).not.toContain(stale);
108     // Fan-out (6) + tail (20m) created, capped well under the 20 limit.
109     expect(script.triggers.length).toBeGreaterThanOrEqual(6);
110     expect(script.triggers.length).toBeLessThanOrEqual(12);
111     expect(lock.releaseLock).toHaveBeenCalled();
112 });
113
114 it("does not search Gmail when the daily cap is already reached", () => {
115     const props = baseProps({
116         [PROP_FILES_ADDED_TODAY]: JSON.stringify({
117             added: MAX_FILES_ADD_PER_DAY,
118             date: nowIso(),
119         }),
120     });
121     install(props);
122
123     syncUserDrives();
124
125     expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
126 });
127
128 it("resets the daily counter on a new calendar day", () => {
129     const yesterday = new Date(Date.now() - ONE_DAY).toJSON();
130     const props = baseProps({
131         [PROP_FILES_ADDED_TODAY]: JSON.stringify({
132             added: MAX_FILES_ADD_PER_DAY,
133             date: yesterday,
134         }),
135     });
136     install(props);
137
138     syncUserDrives();
139
140     // Counter was reset, so the sync loop ran and persisted added: 0.
141     expect((g.GmailApp as { search: jest.Mock }).search).toHaveBeenCalled();
142     const persisted = JSON.parse(props.store[PROP_FILES_ADDED_TODAY]);
143     expect(persisted.added).toBe(0);
144 });
145
146 it("persists the ids of messages it copied for cross-run dedup", () => {
147     const props = baseProps();
148     install(props);
149
150     // A minimal Drive + Gmail wiring so one attachment is copied.
151     const iter = <T>(items: T[]) => {
152         let i = 0;
153         return { hasNext: () => i < items.length, next: () => items[i++] };
154     };
155     const head = {
156         getName: () => "GmailAttachments",
157         getFolders: () => iter([]),
158         createFolder: () => head,
159         getFilesByName: () => iter([]),
160         createFile: () => ({ setDescription: () => undefined }),
161     };
162     g.DriveApp = { getFolders: () => iter([]), createFolder: () => head };
163     const message = {
164         getId: () => "msg-xyz",
165         getFrom: () => "alice@example.com",
166         getDate: () => new Date(2012, 5, 6),

```

```

167         getAttachments: () => [
168             {
169                 getContentType: () => "application/pdf",
170                 getName: () => "report.pdf",
171                 copyBlob: () => ({ setName: () => undefined }),
172             },
173         ],
174     };
175     let firstCall = true;
176     (g.GmailApp as { search: jest.Mock }).search = jest.fn(() => {
177         // Return the thread once, then nothing so the loop converges.
178         if (firstCall) {
179             firstCall = false;
180             return [{ getMessages: () => [message] }];
181         }
182         return [];
183     });
184     syncUserDrives();
185
186     expect(JSON.parse(props.store[PROP_PROCESSED_IDS])).toContain("msg-xyz");
187 });
188 });
189
190

```

30. user (2026-06-10T16:14:58.089Z)

```

1      /*
2      * Installs no-op / jest.fn() stand-ins for the Apps Script globals so modules
3      * that reference them can be imported in a plain Node test environment. Pure
4      * logic modules (query, naming, contentTypes, quota) do not need these, but the
5      * shells (drive, sync, ui, state) do.
6      */
7
8      const g = globalThis as unknown as Record<string, unknown>;
9
10     g.Logger = { log: jest.fn() };
11
12     g.Utilities = { sleep: jest.fn() };
13
14     // Individual tests override these with richer fakes as needed.
15     g.PropertiesService = {
16         getUserProperties: jest.fn(),
17         getScriptProperties: jest.fn(),
18     };
19     g.DriveApp = {
20         getFolders: jest.fn(),
21         createFolder: jest.fn(),
22     };
23     g.GmailApp = { search: jest.fn(() => []), getMessageById: jest.fn() };
24     g.Gmail = {
25         Users: {
26             getProfile: jest.fn(() => ({ historyId: "1" })),
27             History: { list: jest.fn(() => ({ history: [], historyId: "1" }) ) },
28         },
29     };
30     g.ScriptApp = {
31         newTrigger: jest.fn(() => ({
32             timeBased: () => ({ after: () => ({ create: jest.fn() }) })),
33         })),
34         getProjectTriggers: jest.fn(() => []),
35         deleteTrigger: jest.fn(),
36     };
37     g.HtmlService = { createHtmlOutputFromFile: jest.fn() };
38     g.LockService = {

```

```

39     getUserLock: jest.fn(() => ({
40         tryLock: jest.fn(() => true),
41         releaseLock: jest.fn(),
42     })),
43 };
44 g.MailApp = { sendEmail: jest.fn() };
45 g.Session = {
46     getEffectiveUser: jest.fn(() => ({ getEmail: () => "me@example.com" })),
47 };
48

```

31. user (2026-06-10T16:15:05.217Z)

```

1     // Enable the opt-in incremental path for this suite only.
2     jest.mock("../src/config", () => ({
3         ...jest.requireActual("../src/config"),
4         ENABLE_INCREMENTAL_SYNC: true,
5     }));
6
7     import {
8         ONE_DAY,
9         PROP_DEBUG,
10        PROP_FILES_ADDED_TODAY,
11        PROP_FOLDER,
12        PROP_HISTORY_ID,
13        PROP_LAST_SYNCED,
14        PROP_PROCESSED_IDS,
15    } from "../src/config";
16    import { syncUserDrives } from "../src/sync";
17
18    type Store = Record<string, string>;
19    const g = globalThis as unknown as Record<string, unknown>;
20    const nowIso = () => new Date().toJSON();
21
22    function fakeProps(initial: Store) {
23        const store: Store = { ...initial };
24        return {
25            store,
26            getProperty: (k: string) => (k in store ? store[k] : null),
27            setProperty: (k: string, v: string) => {
28                store[k] = String(v);
29            },
30            deleteProperty: (k: string) => {
31                delete store[k];
32            },
33            deleteAllProperties: () => {
34                for (const k of Object.keys(store)) delete store[k];
35            },
36        };
37    }
38
39    function fakeScriptApp() {
40        const triggers: { getHandlerFunction: () => string }[] = [];
41        return {
42            newTrigger: (handler: string) => ({
43                timeBased: () => ({
44                    after: () => ({
45                        create: () => triggers.push({ getHandlerFunction: () => handler }),
46                    }),
47                }),
48            }),
49            getProjectTriggers: () => [...triggers],
50            deleteTrigger: () => undefined,
51        };
52    }

```

```

53
54 function iter<T>(items: T[]) {
55     let i = 0;
56     return { hasNext: () => i < items.length, next: () => items[i++] };
57 }
58
59 function installBase(props: ReturnType<typeof fakeProps>) {
60     g.PropertiesService = { getUserProperties: () => props };
61     g.ScriptApp = fakeScriptApp();
62     g.LockService = {
63         getUserLock: () => ({ tryLock: () => true, releaseLock: jest.fn() }),
64     };
65     g.GmailApp = { search: jest.fn(() => []), getMessageById: jest.fn() };
66 }
67
68 function baseProps(overrides: Store = {}) {
69     return fakeProps({
70         [PROP_DEBUG]: "1",
71         [PROP_FOLDER]: "byContentType",
72         [PROP_LAST_SYNCED]: nowIso(),
73         [PROP_FILES_ADDED_TODAY]: JSON.stringify({ added: 0, date: nowIso() }),
74         ...overrides,
75     });
76 }
77
78 describe("syncUserDrives ? incremental mode", () => {
79     it("processes messages added since the stored historyId and advances it", () => {
80         const props = baseProps({ [PROP_HISTORY_ID]: "100" });
81         installBase(props);
82
83         // A Drive head folder that accepts one file.
84         const head = {
85             getName: () => "GmailAttachments",
86             getFolders: () => iter([]),
87             createFolder: () => head,
88             getFilesByName: () => iter([]),
89             createFile: () => ({ setDescription: () => undefined }),
90         };
91         g.DriveApp = { getFolders: () => iter([]), createFolder: () => head };
92
93         const message = {
94             getId: () => "m1",
95             getFrom: () => "alice@example.com",
96             getDate: () => new Date(2012, 5, 6),
97             getAttachments: () => [
98                 {
99                     getContentType: () => "application/pdf",
100                     getName: () => "report.pdf",
101                     copyBlob: () => ({ setName: () => undefined }),
102                 },
103             ],
104         };
105         (g.GmailApp as { getMessageById: jest.Mock }).getMessageById = jest.fn(
106             () => ({ getThread: () => ({ getId: () => "t1", getMessages: () => [message] })
107         ));
108         g.Gmail = {
109             Users: {
110                 getProfile: jest.fn(),
111                 History: {
112                     list: jest.fn(() => ({
113                         history: [{ messagesAdded: [{ message: { id: "m1" } } ] }]),
114                     historyId: "200",
115                 })),
116             },

```

```

117     },
118 };
119
120 syncUserDrives();
121
122 // Never falls back to a date-window search while the history is valid.
123 expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
124 expect(props.store[PROP_HISTORY_ID]).toBe("200");
125 expect(JSON.parse(props.store[PROP_PROCESSED_IDS])).toContain("m1");
126 });
127
128 it("clears the cursor and rewinds lastSynced when the history expired", () => {
129     const props = baseProps({ [PROP_HISTORY_ID]: "1" });
130     installBase(props);
131     g.Gmail = {
132         Users: {
133             getProfile: jest.fn(),
134             History: {
135                 list: jest.fn(() => {
136                     throw new Error("call failed with error: 404 not found");
137                 }),
138             },
139         },
140     };
141
142     syncUserDrives();
143
144     expect(PROP_HISTORY_ID in props.store).toBe(false);
145     // lastSynced rewound to roughly 30 days ago for the recovery re-scan.
146     const lastSynced = Date.parse(props.store[PROP_LAST_SYNCED]);
147     expect(Date.now() - lastSynced).toBeGreaterThan(29 * ONE_DAY);
148     // No history ? no incremental processing this run; backfill resumes next run.
149     expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
150 });
151
152 it("captures the mailbox historyId once backfill catches up", () => {
153     // No stored historyId ? backfill mode; lastSynced=now means we are already
154     // caught up, so the run captures the current historyId for next time.
155     const props = baseProps();
156     installBase(props);
157     g.Gmail = {
158         Users: {
159             getProfile: jest.fn(() => ({ historyId: "555" })),
160             History: { list: jest.fn() },
161         },
162     };
163
164     syncUserDrives();
165
166     expect((g.GmailApp as { search: jest.Mock }).search).toHaveBeenCalled();
167     expect(props.store[PROP_HISTORY_ID]).toBe("555");
168 });
169 });
170

```

32. user (2026-06-10T16:15:05.403Z)

```

1     import { ONE_DAY, THIRTY_DAYS } from "../src/config";
2     import { buildSearchQuery } from "../src/query";
3
4     // Mirror the function's own local-time day formatting so these assertions are
5     // independent of the machine's timezone.
6     function day(d: Date): string {
7         return `${d.getFullYear()}/${d.getMonth() + 1}/${d.getDate()}`;
8     }

```

```

9
10 describe("buildSearchQuery", () => {
11     it("always includes the base Gmail filters", () => {
12         const r = buildSearchQuery(new Date(2012, 0, 1).toJSON(), new Date(2020, 0, 1));
13         expect(r.query).toContain("to:me in:all has:attachment -in:trash -in:spam");
14     });
15
16     it("advances a 30-day window when not yet caught up", () => {
17         const now = new Date(2020, 0, 1, 12, 0, 0);
18         const lastSynced = new Date(2012, 0, 1).toJSON();
19         const r = buildSearchQuery(lastSynced, now);
20
21         expect(r.isToday).toBe(false);
22         const after = new Date(Date.parse(lastSynced) - ONE_DAY);
23         const before = new Date(Date.parse(lastSynced) + THIRTY_DAYS + ONE_DAY);
24         expect(r.query).toContain("after:" + day(after));
25         expect(r.query).toContain("before:" + day(before));
26     });
27
28     it("clamps the upper bound to now and flags isToday once caught up", () => {
29         const now = new Date(2012, 0, 10, 12, 0, 0);
30         const lastSynced = new Date(2012, 0, 5).toJSON();
31         const r = buildSearchQuery(lastSynced, now);
32
33         expect(r.isToday).toBe(true);
34         const before = new Date(now.getTime() + ONE_DAY);
35         expect(r.query).toContain("before:" + day(before));
36     });
37
38     it("collapses to a single 'after:' clause when the window is one day", () => {
39         const now = new Date(2012, 0, 10, 12, 0, 0);
40         // lastSynced ahead of now makes beforeQ === afterQ.
41         const lastSynced = new Date(now.getTime() + 2 * ONE_DAY).toJSON();
42         const r = buildSearchQuery(lastSynced, now);
43
44         expect(r.isToday).toBe(true);
45         expect(r.query).not.toContain("before:");
46         expect(r.query).toContain("after:");
47     });
48 });
49

```

33. user (2026-06-10T16:15:11.340Z)

```

1     import {
2         getCurrentHistoryId,
3         listAddedSince,
4         threadsForMessageIds,
5     } from "../src/gmailHistory";
6
7     const g = globalThis as unknown as Record<string, unknown>;
8
9     describe("getCurrentHistoryId", () => {
10         it("returns the profile's historyId", () => {
11             g.Gmail = {
12                 Users: { getProfile: () => ({ historyId: "99" }), History: { list: jest.fn() } },
13             };
14             expect(getCurrentHistoryId()).toBe("99");
15         });
16
17         it("returns null when the profile has no historyId", () => {
18             g.Gmail = {
19                 Users: { getProfile: () => ({}), History: { list: jest.fn() } },
20             };
21             expect(getCurrentHistoryId()).toBeNull();
22         });
23     });
24

```

```

22     });
23   });
24
25   describe("listAddedSince", () => {
26     it("pages through results, returning collected ids and the latest historyId", () => {
27       const pages = [
28         {
29           history: [{ messagesAdded: [{ message: { id: "m1" } } ] }],
30           historyId: "100",
31           nextPageToken: "p2",
32         },
33         {
34           history: [{ messagesAdded: [{ message: { id: "m2" } } ] }],
35           historyId: "200",
36         },
37       ];
38       let call = 0;
39       const list = jest.fn(() => pages[call++]);
40       g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
41
42       expect(listAddedSince("50")).toEqual({
43         messageIds: ["m1", "m2"],
44         historyId: "200",
45       });
46       expect(list).toHaveBeenCalledTimes(2);
47     });
48
49     it("returns null when the start historyId has expired", () => {
50       const list = jest.fn(() => {
51         throw new Error("call failed with error: 404 not found");
52       });
53       g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
54       expect(listAddedSince("1")).toBeNull();
55     });
56
57     it("rethrows errors that are not history expiry", () => {
58       const list = jest.fn(() => {
59         throw new Error("backend error 500");
60       });
61       g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
62       expect(() => listAddedSince("1")).toThrow("500");
63     });
64   });
65
66   describe("threadsForMessageIds", () => {
67     it("resolves distinct threads and skips messages that no longer exist", () => {
68       const threadA = { getId: () => "tA" };
69       const byId: Record<string, { getThread: () => { getId: () => string } }> = {
70         a: { getThread: () => threadA },
71         b: { getThread: () => threadA }, // same thread as a
72         c: { getThread: () => ({ getId: () => "tC" }) },
73       };
74       g.GmailApp = { getMessageById: (id: string) => byId[id] ?? null };
75
76       const threads = threadsForMessageIds(["a", "b", "c", "gone"]);
77       expect(threads.map((t) => (t as { getId: () => string }).getId()).sort()).toEqual(
78         ["tA", "tC"],
79       );
80     });
81   });
82

```

34. user (2026-06-10T16:15:12.319Z)

```

1   import { updateDrive } from "../src/drive";

```



```

2
3 // ---- Minimal fakes for the Drive/Gmail objects updateDrive touches ----
4
5 function iterator<T>(items: T[]) {
6     let i = 0;
7     return { hasNext: () => i < items.length, next: () => items[i++] };
8 }
9
10 class FakeFile {
11     description = "";
12     constructor(public name: string) {}
13     setDescription(d: string) {
14         this.description = d;
15         return this;
16     }
17 }
18
19 class FakeFolder {
20     files: FakeFile[] = [];
21     subfolders: FakeFolder[] = [];
22     constructor(public name: string) {}
23     getName() {
24         return this.name;
25     }
26     getFolders() {
27         return iterator(this.subfolders);
28     }
29     createFolder(n: string) {
30         const f = new FakeFolder(n);
31         this.subfolders.push(f);
32         return f;
33     }
34     getFilesByName(n: string) {
35         return iterator(this.files.filter((f) => f.name === n));
36     }
37     createFile(blob: { _name: string }) {
38         const f = new FakeFile(blob._name);
39         this.files.push(f);
40         return f;
41     }
42 }
43
44 function fakeAttachment(name: string, contentType: string) {
45     return {
46         getName: () => name,
47         getContentType: () => contentType,
48         copyBlob: () => ({
49             _name: name,
50             setName(n: string) {
51                 this._name = n;
52                 return this;
53             },
54         })),
55     };
56 }
57
58 let nextMsgId = 0;
59 function fakeThread(
60     from: string,
61     date: Date,
62     attachments: unknown[],
63     id?: string,
64 ) {
65     const messageId = id ?? `msg-${nextMsgId++}`;
66     const message = {

```

```

67         getId: () => messageId,
68         getFrom: () => from,
69         getDate: () => date,
70         getAttachments: () => attachments,
71     };
72     return { getMessages: () => [message] };
73 }
74
75 let head: FakeFolder;
76
77 beforeEach(() => {
78     head = new FakeFolder("GmailAttachments");
79     (globalThis as unknown as Record<string, unknown>).DriveApp = {
80         getFolders: () => iterator<FakeFolder>([]),
81         createFolder: () => head,
82     };
83 });
84
85 const date = new Date(2012, 5, 6, 9, 5, 3);
86
87 describe("updateDrive", () => {
88     it("copies an allowed attachment into the right structure folder", () => {
89         const threads = [
90             fakeThread("alice@example.com", date, [
91                 fakeAttachment("report.pdf", "application/pdf"),
92             ]),
93         ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
94
95         const result = updateDrive(threads, "byContentType", 10);
96
97         expect(result.addedFiles).toBe(1);
98         expect(result.hasError).toBe(false);
99         const bucket = head.subfolders.find((f) => f.name === "Application Data");
100         expect(bucket?.files).toHaveLength(1);
101     });
102
103     it("skips disallowed content types", () => {
104         const threads = [
105             fakeThread("alice@example.com", date, [
106                 fakeAttachment("evil.exe", "application/x-msdownload"),
107             ]),
108         ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
109
110         expect(updateDrive(threads, "byContentType", 10).addedFiles).toBe(0);
111     });
112
113     it("does not copy the same file twice within a run", () => {
114         const threads = [
115             fakeThread("alice@example.com", date, [
116                 fakeAttachment("report.pdf", "application/pdf"),
117                 fakeAttachment("report.pdf", "application/pdf"),
118             ]),
119         ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
120
121         expect(updateDrive(threads, "byContentType", 10).addedFiles).toBe(1);
122     });
123
124     it("stops once maxToAdd files have been written", () => {
125         const threads = [
126             fakeThread("alice@example.com", date, [
127                 fakeAttachment("a.pdf", "application/pdf"),
128                 fakeAttachment("b.pdf", "application/pdf"),
129                 fakeAttachment("c.pdf", "application/pdf"),
130             ]),
131         ] as unknown as GoogleAppsScript.Gmail.GmailThread[];

```

```

132     expect(updateDrive(threads, "byContentType", 2).addedFiles).toBe(2);
133 });
134
135
136 it("reports a fully-handled message id in processedIds", () => {
137     const threads = [
138         fakeThread(
139             "alice@example.com",
140             date,
141             [fakeAttachment("report.pdf", "application/pdf")],
142             "m1",
143         ),
144     ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
145
146     expect(updateDrive(threads, "byContentType", 10).processedIds).toEqual([
147         "m1",
148     ]);
149 });
150
151 it("skips messages already in the processed set without touching Drive", () => {
152     const threads = [
153         fakeThread(
154             "alice@example.com",
155             date,
156             [fakeAttachment("report.pdf", "application/pdf")],
157             "m1",
158         ),
159     ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
160
161     const result = updateDrive(
162         threads,
163         "byContentType",
164         10,
165         new Set(["m1"]),
166     );
167
168     expect(result.addedFiles).toBe(0);
169     expect(result.processedIds).toEqual([]);
170     expect(head.subfolders).toHaveLength(0);
171 });
172
173 it("does not mark a message done when the daily cap truncates it", () => {
174     const threads = [
175         fakeThread(
176             "alice@example.com",
177             date,
178             [
179                 fakeAttachment("a.pdf", "application/pdf"),
180                 fakeAttachment("b.pdf", "application/pdf"),
181             ],
182             "m1",
183         ),
184     ] as unknown as GoogleAppsScript.Gmail.GmailThread[];
185
186     // maxToAdd = 1 copies only the first of two attachments.
187     const result = updateDrive(threads, "byContentType", 1, new Set());
188
189     expect(result.addedFiles).toBe(1);
190     expect(result.processedIds).toEqual([]);
191 });
192 });
193

```

```

1  import {
2      PROP_RUN_STATS,
3      PROP_SUMMARY_LAST_SENT,
4  } from "../src/config";
5  import { maybeSendDailySummary } from "../src/notify";
6
7  type Store = Record<string, string>;
8
9  function fakeProps(initial: Store) {
10     const store: Store = { ...initial };
11     return {
12         store,
13         getProperty: (k: string) => (k in store ? store[k] : null),
14         setProperty: (k: string, v: string) => {
15             store[k] = String(v);
16         },
17         deleteAllProperties: () => {
18             for (const k of Object.keys(store)) delete store[k];
19         },
20     };
21 }
22
23 const g = globalThis as unknown as Record<string, unknown>;
24
25 function install(initial: Store) {
26     const props = fakeProps(initial);
27     g.PropertiesService = { getUserProperties: () => props };
28     return props;
29 }
30
31 const priorDayStats = JSON.stringify({
32     day: "2012-06-05",
33     runs: 3,
34     filesAdded: 7,
35     errors: 0,
36     lastError: "",
37 });
38
39 beforeEach(() => {
40     (g.MailApp as { sendEmail: jest.Mock }).sendEmail.mockClear();
41 });
42
43 describe("maybeSendDailySummary", () => {
44     const now = new Date(2012, 5, 6, 9, 0, 0); // 2012-06-06
45
46     it("is a no-op when already summarised today", () => {
47         install({
48             [PROP_SUMMARY_LAST_SENT]: "2012-06-06",
49             [PROP_RUN_STATS]: priorDayStats,
50         });
51
52         maybeSendDailySummary(now);
53
54         expect((g.MailApp as { sendEmail: jest.Mock }).sendEmail).not.toHaveBeenCalled();
55     });
56
57     it("summarises the prior day and marks today sent", () => {
58         const props = install({ [PROP_RUN_STATS]: priorDayStats });
59
60         maybeSendDailySummary(now);
61
62         // Email stays off by default (ENABLE_SUMMARY_EMAIL=false), but the marker
63         // advances so we don't summarise the same rollover twice.
64         expect(props.store[PROP_SUMMARY_LAST_SENT]).toBe("2012-06-06");
65         expect((g.MailApp as { sendEmail: jest.Mock }).sendEmail).not.toHaveBeenCalled();

```

```

66     });
67
68     it("does not summarise when the only stats are for today", () => {
69         const todayStats = JSON.stringify({
70             day: "2012-06-06",
71             runs: 1,
72             filesAdded: 0,
73             errors: 0,
74             lastError: "",
75         });
76         const props = install({ [PROP_RUN_STATS]: todayStats });
77
78         maybeSendDailySummary(now);
79
80         // Nothing from a prior day; still arms the marker for the next rollover.
81         expect(props.store[PROP_SUMMARY_LAST_SENT]).toBe("2012-06-06");
82     });
83 });
84

```

36. user (2026-06-10T16:15:17.454Z)

```

1     import { readFileSync, readdirSync } from "fs";
2     import { join } from "path";
3
4     /*
5      * Enforces the auditability invariant (docs/ARCHITECTURE.md): this repo is the
6      * only code that touches a user's Gmail, and it makes NO network egress today.
7      *
8      * The ONLY sanctioned way data may ever leave a user's account is a deliberate,
9      * reviewed REDACT/MINIMIZE -> user's-own-endpoint path. When that lands, update
10     * this guard to assert the single allowlisted destination ? do not delete it.
11     */
12
13     // Apps Script's only outbound-network primitive.
14     const FORBIDDEN = /UrlFetchApp/;
15
16     function tsFiles(dir: string): string[] {
17         return readdirSync(dir, { withFileTypes: true }).flatMap((entry) => {
18             const full = join(dir, entry.name);
19             if (entry.isDirectory()) return tsFiles(full);
20             return entry.name.endsWith(".ts") ? [full] : [];
21         });
22     }
23
24     describe("auditability invariant", () => {
25         it("src/ makes no outbound network calls (no UrlFetchApp)", () => {
26             const offenders = tsFiles("src").filter((file) =>
27                 FORBIDDEN.test(readFileSync(file, "utf8")),
28             );
29             expect(offenders).toEqual([]);
30         });
31     });
32

```

37. user (2026-06-10T16:15:25.087Z)

```

1     # Architecture
2
3     > Status: living document. Captures the target architecture for the
4     > extraction pivot and the decisions behind it. Product framing is in
5     > [VISION.md](./VISION.md). This supersedes the original "just copy
6     > attachments" mental model; the copier survives as one extractor at the leaf.
7
8     ## Auditability invariant (binding)

```

```

9
10  **This repository is the _only_ component that reads or processes a user's
11  Gmail, and it must remain auditable and self-deployable by a power user at any
12  time.** This is the product's core privacy guarantee and a deliberate constraint
13  on every future decision (see ADR-008). It is why the project is built to be
14  open-sourced: *don't trust us ? read, and run, the code.*
15
16  Every change MUST preserve all of the following. A change that cannot is, by
17  definition, a decision to break the invariant and needs explicit owner sign-off.
18
19  1. **Sole Gmail surface.** Gmail is read/processed *only* here, in the user's own
20  account (`executeAs: USER_ACCESSING`). No other component ? Muneem included ?
21  ever holds the Gmail OAuth grant or sees raw mail. Muneem may *orchestrate*
22  this app; it is never *itself* the Gmail reader.
23  2. **Egress == the audited boundary.** Nothing derived from a user's mail leaves
24  their Google account except through the in-repo **REDACT/MINIMIZE** stage,
25  carrying only minimized, non-PII signal, only to the user's own hosted
26  service, never to third parties. The set of egress destinations is explicit
27  and reviewable in source.
28  3. **Run-your-own-from-source.** A power user can build and deploy their own
29  instance from the published source and obtain an equivalent app; the build is
30  reproducible and the verification steps are documented ? "trust by running the
31  code you read," not merely "read the code."
32  4. **No secrets in source.** Credentials exist only as deploy-time secrets; the
33  repository contains nothing sensitive.
34  5. **Transparent over clever** at the read and egress/redaction boundaries ?
35  prefer obviously-correct, reviewable code even at some cost to brevity.
36
37  **PR review checklist (apply to every change):**
38
39  - [ ] Gmail access stays entirely within this repo, in-account?
40  - [ ] If any data leaves the account, does it pass through REDACT/MINIMIZE and go
41  only to the user's own service (non-PII)?
42  - [ ] Build still reproducible and self-deployable from source?
43  - [ ] No new secrets, third-party endpoints, or network calls? (Default: none ?
44  enforced by `tests/auditability.test.ts`.)
45
46  ## Principles (preserved from today's codebase)
47
48  1. **Shells around pure functions.** GAS-facing code is a thin shell; all logic
49  is pure and unit-tested.
50  2. **Quota discipline.** Respect the 6-minute execution limit, the 90-min/day
51  trigger budget, and per-user caps. Self-healing trigger fan-out; per-user
52  lock; "mark done only on clean completion."
53  3. **Per-user isolation.** Runs as the accessing user; state is per-user.
54  4. **Versioning & provenance everywhere.** Every persisted shape carries a
55  `schemaVersion`; every extracted entity carries where it came from and which
56  extractor (and version) produced it.
57
58  ## The pipeline
59
60  This repo owns the left half; downstream services own the right half. They meet
61  at the **handoff contract** only.
62
63  ...
64
65  Gmail ???  INGEST ???  CLASSIFY ???  EXTRACT ???  REDACT/MINIMIZE ???  HANDOFF ???  Muneem
66  /
67  (rich      (type +      (normalize (strip PII,      (sink)      grocery
68  /
69  EmailMsg) confidence) light)      encrypt)      coupon
70
71  svc
72
73  ?
74
75  parse,

```

```

69          ???? RAW ARCHIVE (Drive folders ? optional per type later)
encrypted
70          ???? LOCAL INDEX (Store interface; Sheets now, remote later)
store,
71
analytics
72   ``
73
74   - **INGEST** builds a durable `EmailMessage` **once** per message (body, subject,
75     labels, attachment metadata ? via the Gmail advanced service). Extraction
76     re-runs over this record without re-touching Gmail or re-copying blobs.
77   - **CLASSIFY** assigns a type + confidence (sender/subject/body rules now; richer
78     models downstream later).
79   - **EXTRACT** is a registry of pluggable, versioned extractors. The current
80     attachment copier becomes the `RawArchive` extractor.
81   - **REDACT/MINIMIZE** enforces the privacy north star before any egress.
82   - **HANDOFF** routes a normalized payload to the right downstream **sink**.
83
84   ## Data model
85
86   All pure shapes, versioned from day one.
87
88   ``
89   EmailMessage {                                // INGEST output ? the stable substrate
90     schemaVersion, id, threadId, historyId?,
91     from, to[], cc[], subject, date, labels[],
92     snippet, plainBody?, htmlBody?,
93     attachments: [{ index, name, contentType, sizeBytes,
94                     attachmentId/partId, sha256?, driveFileId? }],
95   }
96
97   ExtractedEntity {                             // EXTRACT output ? the queryable payload
98     schemaVersion, entityKey,                   // stable dedup key, e.g.
99                                           // receipt:{merchant}:{total}:{date}
100    type, subtype, confidence,
101    fields,                                     // typed per kind (receipt, statement, ?)
102    provenance: { sourceMsgIds[], attachmentIndex?,
103                  extractor, extractorVersion, extractedAt, model? },
104    status: pending|extracted|low_confidence|needs_review|corrected,
105    links: { driveFileId?, ? },
106  }
107
108  HandoffPayload {                               // REDACT/MINIMIZE output ? what leaves
109    schemaVersion, entityKey, type, extractorVersion,
110    nonPiiFields,                               // minimized, scrubbed signal only
111    // encrypted in transit; PII stripped or tokenized
112  }
113  ``
114
115  - **Entity keys** give content-stable dedup (re-mailed receipts; cross-source
116    fusion of an email confirmation + a PDF ticket ? one entity).
117  - **Attachment `sha256`** enables content-addressable dedup of re-sent files.
118  - **Provenance + `extractorVersion`** make "this extraction is stale ? re-run"
119    possible and auditable.
120
121  ## The extraction ? processing boundary
122
123  - This repo produces a normalized, classified, **redacted** `HandoffPayload` per
124    detected document/entity and routes it to a downstream **sink**.
125  - Downstream services (Muneem for finance; siblings for groceries/coupons) own
126    domain parsing, indexing, **encrypted storage**, and analytics.
127  - **Contract is owned jointly, defined from our side first.** The exact
128    Muneem-facing schema and transport are **TBD and aligned with that repo**
129    (this repo's GitHub tooling is scoped to itself, so the contract is specified
130    here and reconciled with Muneem separately).

```

```

131 - Hard rule: this repo imports **no** domain-parsing logic; downstream reads
132 **no** Gmail.
133
134 ## Storage abstraction (decision: extensible, not baked-in)
135
136 A single `Store` / `Sink` interface; code depends only on the interface.
137
138 | Impl | When | Properties |
139 | --- | --- | --- |
140 | `GoogleSheetsStore` (user-owned "LifeIndex" sheet) | v1 | in-account, queryable, zero
infra, doubles as a first dashboard |
141 | `RemoteStore` (our hosted DB) / Muneem-API sink | later | the durable system of record
|
142
143 `PropertiesService` stays only for small per-user **control state** (cursors,
144 flags, counters) ? never the entity index.
145
146 ## Privacy & egress controls
147
148 - **Egress only to our own isolated, hosted service. No third parties.**
149 - A **Redactor/Minimizer** stage runs before any egress: strip/tokenize PII,
150 keep only non-PII signal, encrypt in transit (encrypted at rest downstream).
151 - North star: continuously **shrink** what leaves while keeping the product
152 healthy.
153 - The in-account tier (Phases 0?2) has **no egress** and needs no privacy
154 change. The external tier (Phase 3) is **flag-gated + separately consented**,
155 and updates the privacy policy to name exactly what is sent and where.
156 - **Cascade to be aware of:** sending restricted-scope Gmail data outside Google
157 is what makes Google's **CASA security assessment mandatory for a public
158 launch**. Deferred ? consistent with "stay <100 until the user base justifies
159 the investment."
160 - **Open tension to resolve (Phase 3 design):** bank-statement and receipt
161 signal (merchant, amounts, account identifiers) is often itself sensitive.
162 "Non-PII only" needs a concrete per-vertical definition of what is sent vs.
163 tokenized vs. kept fully in-account.
164
165 ## State & cursors
166
167 - Keep the hybrid `history.list` + date-window skeleton; **extend, don't
168 duplicate.**
169 - Add an **extraction-aware cursor** ("fully extracted up to X with
170 extractor-set V") distinct from the copy cursor, plus per-message extraction
171 status ? so a "re-extract window for new rules" reprocesses without
172 re-copying (content hash) or duplicating entities (entity keys).
173
174 ## Quota, cost & pacing
175
176 - **Separate budgets:** raw-copy cap (existing) vs. extraction cap vs.
177 egress/handoff cap.
178 - Per-item time budgets, **prioritization** (recent + high-confidence/trusted
179 senders first), and **deferral** of heavy items.
180 - Heavy ML/PDF work happens **downstream**, not in Apps Script ? GAS stays the
181 lightweight, reliable watcher/archiver/handoff.
182
183 ## Observability
184
185 - Per-**classifier** and per-**extractor** success / failure / ambiguity rates ?
186 not just "files added."
187 - Surfaced **review queues:** "matched no extractor" and "low confidence."
188 - Structured logs carrying `messageId`; extraction & handoff metrics.
189
190 ## Platform spectrum
191
192 1. **Pure-GAS conservative extractors** (regex, known formats `.ics`/`.csv`,
193 native Drive OCR via Doc conversion) for the easy ~70%, **in-account**.

```



```

194 2. **Hosted, isolated service** (Muneem pattern) for heavy/ML extraction,
195     storage, and analytics ? fed only the minimized, non-PII handoff.
196
197 ## Decision log
198
199 | # | Decision | Rationale |
200 | --- | --- | --- |
201 | ADR-001 | Pipeline (ingest ? classify ? extract ? redact ? handoff), not a copy loop |
Treat mail as structured data; make extraction re-runnable over a durable record |
202 | ADR-002 | Extraction (this repo) is decoupled from processing (Muneem & siblings) |
Isolate domain logic + encrypted storage + analytics downstream; clean contract seam |
203 | ADR-003 | Storage via a `Store`/`Sink` interface ? Sheets v1, remote/Muneem later |
Extensible by design; never baked into the code |
204 | ADR-004 | Egress: non-PII only, to our own isolated hosted service, no third parties;
scrub + encrypt; minimize | Privacy north star; preserves trust; in-account baseline needs no
egress |
205 | ADR-005 | Audience: <100 (Testing) now, public later; CASA deferred | Harden + learn
value before investing in assessment/infra |
206 | ADR-006 | First verticals: grocery receipts + bank statements (? Muneem) | Volume +
the high-fidelity handoff path |
207 | ADR-007 | Preserve current strengths (pure/shell, quotas, triggers, per-user,
versioning/provenance) | The recent rewrite is exactly why this pivot is feasible |
208 | ADR-008 | **Option A:** this repo is the sole Gmail-touching component ? in-account,
auditable, self-deployable; Muneem orchestrates but never holds the Gmail grant. Codified as the
binding [auditability invariant](#auditability-invariant-binding). | Verifiable privacy is the
product's moat and shrinks future verification scope; it concentrates the trust-critical surface
in one open, in-account place. |
209
210 ## Open items for the next iteration
211
212 - **Muneem handoff contract:** schema + transport (e.g. `UrlFetch` to a signed
213     endpoint vs. write-to-a-shared-store) ? define jointly with `avidullu/muneem`.
214 - **Per-vertical "non-PII" definition** for receipts and bank statements (what is
215     sent vs. tokenized vs. kept in-account).
216 - **First extractor slice** to build in Phase 1 (recommend a narrow
217     "receipt-smelling" detector) and its golden-email test fixtures.
218

```

38. user (2026-06-10T16:15:32.172Z)

```

1  import {
2      MAX_FILES_ADD_PER_DAY,
3      ONE_YEAR,
4      PROP_DEBUG,
5      PROP_FILES_ADDED_TODAY,
6      PROP_FOLDER,
7      PROP_LAST_SYNCED,
8  } from "../src/config";
9  import { doGet, processInput, type SetupForm } from "../src/ui";
10
11  type Store = Record<string, string>;
12
13  function fakeProps() {
14      const store: Store = { stale: "x" };
15      return {
16          store,
17          getProperty: (k: string) => (k in store ? store[k] : null),
18          setProperty: (k: string, v: string) => {
19              store[k] = String(v);
20          },
21          deleteAllProperties: () => {
22              for (const k of Object.keys(store)) delete store[k];
23          },
24      };
25  }

```

```

26
27 const g = globalThis as unknown as Record<string, unknown>;
28 let props: ReturnType<typeof fakeProps>;
29 let created: string[];
30
31 beforeEach(() => {
32     props = fakeProps();
33     created = [];
34     g.PropertiesService = { getUserProperties: () => props };
35     g.ScriptApp = {
36         newTrigger: (handler: string) => ({
37             timeBased: () => ({ after: () => ({ create: () => created.push(handler) }) })),
38         },
39         getProjectTriggers: () => [],
40         deleteTrigger: jest.fn(),
41     };
42 });
43
44 describe("processInput", () => {
45     it("resets state, stores preferences and schedules the first run", () => {
46         const result = processInput({ folderStructure: "byMonthly", syncFrom: "2" });
47
48         // Stale state cleared, new preferences written.
49         expect(props.store.stale).toBeUndefined();
50         expect(props.store[PROP_FOLDER]).toBe("byMonthly");
51         expect(props.store[PROP_DEBUG]).toBe("1");
52         expect(props.store[PROP_FILES_ADDED_TODAY]).toBeDefined();
53
54         // lastSynced is roughly two years back.
55         const lastSynced = Date.parse(props.store[PROP_LAST_SYNCED]);
56         const expected = Date.now() - 2 * ONE_YEAR;
57         expect(Math.abs(lastSynced - expected)).toBeLessThan(60_000);
58
59         // First background run scheduled.
60         expect(created).toContain("syncUserDrives");
61
62         // The success message reflects the real cap (fixes the old 240-vs-250 bug).
63         expect(result).toContain(String(MAX_FILES_ADD_PER_DAY));
64         expect(result).not.toContain("250");
65     });
66
67     it("rejects an unknown folder structure without touching state", () => {
68         const result = processInput({ folderStructure: "bogus", syncFrom: "2" });
69
70         expect(result).toContain("folder structure");
71         // Nothing was scheduled or written.
72         expect(created).not.toContain("syncUserDrives");
73         expect(props.store[PROP_FOLDER]).toBeUndefined();
74         expect(props.store.stale).toBe("x");
75     });
76
77     it.each(["0", "11", "abc", ""])(
78         "rejects an out-of-range / non-numeric syncFrom (%s)",
79         (syncFrom) => {
80             const result = processInput({ folderStructure: "byYear", syncFrom });
81
82             expect(result).toContain("years");
83             expect(created).not.toContain("syncUserDrives");
84             expect(props.store[PROP_LAST_SYNCED]).toBeUndefined();
85         },
86     );
87
88     it("handles a missing/empty form object without throwing", () => {
89         const result = processInput(undefined as unknown as SetupForm);
90

```

```

91     expect(result).toContain("folder structure");
92     expect(created).not.toContain("syncUserDrives");
93 });
94
95 it("returns a clear permission message when setup fails (declined scope)", () => {
96     // Simulate the user declining the script.scriptapp scope on the granular
97     // consent screen: any ScriptApp call then throws.
98     g.ScriptApp = {
99         newTrigger: () => {
100             throw new Error("ScriptApp not authorized");
101         },
102         getProjectTriggers: () => {
103             throw new Error("ScriptApp not authorized");
104         },
105         deleteTrigger: jest.fn(),
106     };
107
108     const result = processInput({ folderStructure: "byYear", syncFrom: "1" });
109
110     expect(result).toContain("Setup couldn't finish");
111     expect(result).toContain("allow all");
112     // No trigger was scheduled.
113     expect(created).not.toContain("syncUserDrives");
114 });
115 });
116
117 describe("doGet", () => {
118     it("serves the index HTML file", () => {
119         const output = { name: "index-output" };
120         const create = jest.fn(() => output);
121         g.HtmlService = { createHtmlOutputFromFile: create };
122
123         expect(doGet()).toBe(output);
124         expect(create).toHaveBeenCalledWith("index");
125     });
126 });
127

```

39. user (2026-06-10T16:15:32.324Z)

```

1      # Gmail Attachments ? Google Drive
2
3      A Google Apps Script that copies your Gmail attachments into a `GmailAttachments`
4      folder in Google Drive and keeps it updated as new mail arrives. You choose how
5      the attachments are organised (by content type, by month, or by year) and how
6      far back the first sync should reach.
7
8      Originally a 2013 Chrome Web Store app; this is a modernised rebuild on the
9      **V8 runtime** with a TypeScript source, an esbuild build, and a local Jest test
10     suite. The web app runs **as each accessing user** (`executeAs: USER_ACCESSING`),
11     so it can serve just you, a small group, or a Google Workspace org ? each
12     person's attachments are copied within their own account. See
13     [Multi-user deployment / publishing](#multi-user-deployment--publishing).
14
15     > **Where this is headed:** attachment archiving is the hardened core of a larger
16     > goal ? turning email into a structured, queryable record of one's life
17     > (receipts, statements, travel, ?) via an ingest ? classify ? extract ? handoff
18     > pipeline, with domain processing isolated in downstream services. See
19     > [`docs/VISION.md`](./docs/VISION.md) and
20     [`docs/ARCHITECTURE.md`](./docs/ARCHITECTURE.md).
21
22     ## How it works
23
24     - `doGet` serves a setup form (`html/index.html`).
25     - `processInput` stores your choices and schedules the first background run.

```

```

25 - `syncUserDrives` is a self-rescheduling time-based trigger that walks Gmail in
26 30-day windows, copying allowed attachments into Drive while respecting a
27 daily file cap (`MAX_FILES_ADD_PER_DAY`) and a per-run wall-clock budget.
28
29 State lives in `PropertiesService` (user properties). Existing Drive files are
30 never modified ? re-running setup only resets the script's own bookkeeping.
31
32 ## Project layout
33
34 | Path | Purpose |
35 | --- | --- |
36 | `src/` | TypeScript source. Pure logic (`query`, `naming`, `quota`, `contentTypes`) is
separated from the Google-service shells (`drive`, `sync`, `ui`, `state`). |
37 | `src/entry.ts` | Bundle entry; esbuild exposes `doGet` / `processInput` /
`syncUserDrives` as globals. |
38 | `html/index.html` | The setup form (pushed to Apps Script as `index`). |
39 | `appsscript.json` | Manifest: V8 runtime, OAuth scopes, web-app settings. |
40 | `tests/` | Jest unit tests. |
41 | `dist/` | Build output that `clasp` pushes (git-ignored). |
42
43 ## Develop
44
45 ```bash
46 npm install
47 npm run typecheck # tsc --noEmit
48 npm run lint # eslint
49 npm test # jest
50 npm run build # bundle src -> dist/Code.js (+ manifest + html)
51 ```
52
53 ## Deploy (requires your Google account)
54
55 1. `npm install -g @google/clasp && clasp login`
56 2. The project is already linked via the committed `.clasp.json` (`scriptId` +
57 `rootDir: dist`). To target a different project, edit `scriptId`.
58 3. `npm run push` ? builds and `clasp push`es `dist/`.
59 4. In the Apps Script editor, deploy as a **Web app**. The manifest already sets
60 `*Execute as: User accessing*` and `*Who has access: Anyone*`, so each visitor
61 runs against their own account. Open the URL and submit the form.
62 5. Set `timeZone` in `appsscript.json` to your own zone ? the Gmail
63 `before:`/`after:` day boundaries follow it.
64
65 > Using it only for yourself? It still works as-is ? you're simply the only
66 > user. To let others in, follow the runbook below.
67
68 ### Auto-deploy from CI
69
70 Pushing to `master` runs the `deploy` job in `.github/workflows/ci.yml`, which
71 `clasp push`es and `clasp deploy`s the build. It needs a single repository
72 secret:
73
74 - `CLASPRC_JSON` ? the full contents of `~/.clasprc.json` after `clasp login`
75 (generate it from a dedicated owner account; the token refreshes over time).
76
77 The `scriptId` lives in the committed `.clasp.json` (it is not sensitive ? it
78 appears in the web-app URL and grants nothing without the OAuth token). If the
79 secret is unset the job logs a warning and skips, so forks/CI without
80 credentials stay green.
81
82 ### OAuth scopes
83
84 `gmail.readonly` (we only read mail), `drive` (create/enumerate folders and
85 files), and `script.scriptapp` (manage the background triggers).
86
87 ## Observability

```

```

88
89 Each run logs structured, level-prefixed lines (`[INFO]` / `[ERROR]` / `[DEBUG]`)
90 to the Apps Script execution log / Stackdriver, and accumulates per-day stats
91 (runs, files added, errors, last error). On the first run of a new calendar day
92 the previous day's summary is logged.
93
94 ### Optional daily summary email
95
96 Off by default. To receive the daily summary by email:
97
98 1. Set `ENABLE_SUMMARY_EMAIL = true` (and optionally `SUMMARY_EMAIL_RECIPIENT`)
99   in `src/config.ts`.
100
101 2. Add the mail-send scope to `appsscript.json` `oauthScopes`:
102   `https://www.googleapis.com/auth/script.send_mail`
103
104 3. Rebuild and redeploy; you'll be prompted to re-consent to the new
105   "send email as you" permission.
106
107 ## Multi-user deployment / publishing
108
109 The app is already wired for multiple users (runs as the accessing user, with
110 all state in their `getUserProperties`, a per-user lock, and per-user
111 triggers). What it takes to actually let other people in depends on the audience
112 ? and this project deliberately targets the no-verification zone: a small
113 named group and/or a single Workspace org. That avoids Google's brand /
114 restricted-scope verification and the annual paid third-party CASA security
115 assessment that a fully public launch with `gmail.readonly` + `drive` would
116 require.
117
118 ### One-time Google Cloud setup
119
120 1. Use a standard Cloud project (the default Apps Script one can't host a
121   managed consent screen). Create a project in the
122   [Cloud Console](https://console.cloud.google.com/), then in the Apps Script
123   editor: Project Settings ? Google Cloud Platform (GCP) Project ? Change
124   project** and paste the project number. This switch is permanent.
125
126 2. Enable APIs in that project: Gmail API (required by the advanced Gmail
127   service this app uses) and Google Drive API.
128
129 3. Configure the OAuth consent screen / Branding: app name
130   (`Gmail Attachments to Drive`), a user-support email, a developer-contact
131   email, and ? recommended ? a homepage and a privacy policy URL. A ready
132   policy lives at [PRIVACY.md](./PRIVACY.md); host it (e.g. GitHub Pages) and
133   add that domain under Authorized domains.
134
135 ### Pick an audience (same build, different consent config)
136
137 | | Small group | Workspace org |
138 | --- | --- | --- |
139 | User type | External, status Testing | Internal |
140 | Who can use it | Up to 100 Google accounts you add as Test users | Everyone
141   in your Workspace org |
142 | Consent screen | Shows a "Google hasn't verified this app" warning (expected) | No
143   warning |
144 | Caveat | Test users re-consent ~weekly (7-day refresh-token expiry) | Org-only;
145   not usable by external gmail.com accounts |
146 | Verification / CASA | None | None |
147
148 Then deploy (the CI `clasp push`/`deploy` flow is unchanged) and share the
149 `/exec` web-app URL. Note: switching the GCP project / `executeAs` forces
150 everyone ? including you ? to re-authorize once.
151
152 ### Going fully public (out of scope here)
153
154 Letting anyone use it (External + Production) requires brand verification,
155 restricted-scope verification, domain verification, and an annual CASA
156 security assessment** (~$125k+, 6?16 weeks). If you ever go there, first narrow

```

```
150 `drive` ? `drive.file` (the app only touches files it creates) to shrink the
151 review surface ? though `gmail.readonly` remains restricted regardless.
152
153 ## License
154
155 MIT ? see headers in source files.
156
```

40. user (2026-06-10T16:15:41.256Z)

```
1 import {
2   collectAddedMessageIds,
3   isHistoryExpired,
4   nextHistoryId,
5   type HistoryListResponse,
6 } from "../src/historySync";
7
8 describe("collectAddedMessageIds", () => {
9   it("collects unique ids across pages in first-seen order", () => {
10     const responses: HistoryListResponse[] = [
11       {
12         history: [
13           { messagesAdded: [{ message: { id: "a" } }, { message: { id: "b" } }] },
14         ],
15       },
16       {
17         history: [
18           { messagesAdded: [{ message: { id: "b" } }, { message: { id: "c" } }] },
19         ],
20       },
21     ];
22     expect(collectAddedMessageIds(responses)).toEqual(["a", "b", "c"]);
23   });
24
25   it("ignores records without messagesAdded and entries without an id", () => {
26     const responses: HistoryListResponse[] = [
27       { history: [{}, { messagesAdded: [{}, { message: {} }] } ] },
28       {},
29     ];
30     expect(collectAddedMessageIds(responses)).toEqual([]);
31   });
32 });
33
34 describe("nextHistoryId", () => {
35   it("returns the last page's historyId", () => {
36     expect(nextHistoryId([{ historyId: "10" }, { historyId: "20" }], "1")).toBe(
37       "20",
38     );
39   });
40
41   it("skips trailing pages that report no historyId", () => {
42     expect(nextHistoryId([{ historyId: "10" }, {}], "1")).toBe("10");
43   });
44
45   it("falls back to the start id when no page reports one", () => {
46     expect(nextHistoryId([], "7")).toBe("7");
47   });
48 });
49
50 describe("isHistoryExpired", () => {
51   it("detects a 404 details code", () => {
52     expect(isHistoryExpired({ details: { code: 404 } })).toBe(true);
53   });
54
55   it("detects 404 in the error message text", () => {
```

```

56     expect(isHistoryExpired(new Error("call failed with error: 404"))).toBe(
57         true,
58     );
59 });
60
61 it("detects a 'not found' message", () => {
62     expect(isHistoryExpired(new Error("Requested entity was not found."))).toBe(
63         true,
64     );
65 });
66
67 it("returns false for unrelated errors", () => {
68     expect(isHistoryExpired(new Error("rate limit exceeded (429)"))).toBe(false);
69     expect(isHistoryExpired(null)).toBe(false);
70 });
71 });
72

```

41. user (2026-06-10T16:15:41.507Z)

```

1     import { addProcessedId, parseProcessedIds } from "../src/processed";
2
3     describe("parseProcessedIds", () => {
4         it("returns an empty array for null/empty input", () => {
5             expect(parseProcessedIds(null)).toEqual([]);
6             expect(parseProcessedIds("")).toEqual([]);
7         });
8
9         it("parses a JSON array of ids", () => {
10             expect(parseProcessedIds('["a","b"]')).toEqual(["a", "b"]);
11         });
12
13         it("tolerates malformed or non-array JSON as empty", () => {
14             expect(parseProcessedIds("not json")).toEqual([]);
15             expect(parseProcessedIds('{ "a":1 }')).toEqual([]);
16         });
17     });
18
19     describe("addProcessedId", () => {
20         it("appends a new id", () => {
21             expect(addProcessedId(["a"], "b")).toEqual(["a", "b"]);
22         });
23
24         it("is a no-op for an id already present", () => {
25             const ids = ["a", "b"];
26             expect(addProcessedId(ids, "a")).toBe(ids);
27         });
28
29         it("trims to the newest `max` entries, dropping the oldest", () => {
30             expect(addProcessedId(["a", "b", "c"], "d", 3)).toEqual(["b", "c", "d"]);
31         });
32
33         it("keeps insertion order when under the cap", () => {
34             let ids: string[] = [];
35             for (const id of ["a", "b", "c"]) {
36                 ids = addProcessedId(ids, id, 10);
37             }
38             expect(ids).toEqual(["a", "b", "c"]);
39         });
40     });
41

```

42. user (2026-06-10T16:25:40.875Z)

Structured output provided successfully

43. assistant (2026-06-10T16:25:50.853Z)

Review complete. The structured findings have been submitted: 3 high-severity issues (incremental historyId cursor advancing past unprocessed messages, `getMessageById` throwing on deleted messages and wedging incremental sync, backfill advancing `lastSynced` past windows with errors), 10 medium-severity issues spanning the Drive folder lookup, runtime-budget enforcement, history pagination/expiry handling, setup-vs-sync races, naming collisions, logging privacy, and CI deployment, plus 6 genuinely-worth-fixing low items.